DESENVOLVIMENTO DE SISTEMAS JAVA

NOSSA HISTÓRIA

A nossa história inicia com a realização do sonho de um grupo de empresários, em atender à crescente demanda de alunos para cursos de Graduação e Pós-Graduação. Com isso foi criado a nossa instituição, como entidade oferecendo serviços educacionais em nível superior.

A instituição tem por objetivo formar diplomados nas diferentes áreas de conhecimento, aptos para a inserção em setores profissionais e para a participação no desenvolvimento da sociedade brasileira, e colaborar na sua formação contínua. Além de promover a divulgação de conhecimentos culturais, científicos e técnicos que constituem patrimônio da humanidade e comunicar o saber através do ensino, de publicação ou outras normas de comunicação.

A nossa missão é oferecer qualidade em conhecimento e cultura de forma confiável e eficiente para que o aluno tenha oportunidade de construir uma base profissional e ética. Dessa forma, conquistando o espaço de uma das instituições modelo no país na oferta de cursos, primando sempre pela inovação tecnológica, excelência no atendimento e valor do serviço oferecido.

Sumário

INTRODUÇÃO 4

A PROGRAMAÇÃO ORIENTADA A OBJETOS 6

O JAVA 11

ONDE O JAVA ESTÁ PRESENTE..... 14

AS PRINCIPAIS VANTAGENS DO JAVA..... 18

PLATAFORMAS JAVA..... 19

Criando um projeto utilizando a API Eclipse..... 22

A estrutura de um programa feito em Java..... 28

JSF..... 53

PRIMEFACES 54

OS PASSOS PARA A IMPLANTAÇÃO DE UM SISTEMA JAVA..... 55

O FUTURO DO JAVA 59

REFERÊNCIAS..... 91

INTRODUÇÃO

Todos que sentem necessidade de criar um sistema BBB (bom, bonito e barato), e já tem certo conhecimento em programação, na hora de escolher a linguagem para desenvolver a aplicação, deve passar na cabeça em usar o JAVA, que é uma linguagem de programação OPEN Source com bastantes ferramentas de fácil acesso, muito material disponível, com uma qualidade enorme e uma grande gama de componentes muito úteis.

Para quem acabou escolhendo o Java, segue algumas dicas, de situações nas quais muitos desenvolvedores iniciais têm grandes chances passar.

Começando o projeto do sistema, encontram-se certos “probleminhas”, ou melhor, descuido de principiante, com isso o intuito deste artigo é trazer a quem ler isto, uma forma um tanto que informal e com certas dicas ou análises para quem está pensando em começar a desenvolver algo pequeno nesta ferramenta, claro é algo básico, porém em certos casos, acaba passando despercebido, e vale a pena sempre frisar neste quesito.

VÍDEO DE APOIO

Vídeo 1: Curso de Java#02 – Como Funciona o Java – Gustavo Guanabara
Disponível em:< https://www.youtube.com/watch?v=v_ZCtgwbS3o>

Sinopse: Neste vídeo do Canal Curso em Vídeo, Gustavo Guanabara fala s obre Java:

Como funciona o Java? Você sabe o que é JVM, JRE, JDK, JavaC, bytecode? É exatamente isso que você vai aprender durante essa aula.

O JavaC (Java Compiler) vai transformar o Código Fonte Java em Bytecode, um código específico que vai executar em uma Máquina Virtual Java (Java Virtual Machine - JVM).

Uma JVM é uma solução de software capaz de executar bytecode em qualquer máquina, independente da plataforma que o Código Fonte foi compilado. Java é multiplataforma sem a necessidade de recompilação. A JVM é composta por várias partes internas, como por exemplo Loader, Verificador, Interpretador, Gerenciador de Memória e Compilador JIT (Just in Time).

Atualmente existem JVMs em várias plataformas, como Computadores Pessoais (Windows, Linux, MacOS), dispositivos móveis (Android) e até mesmo os modernos wearables (relógios inteligentes, óculos inteligentes).

O princípio de funcionamento desse mecanismo explicado acima é conhecido pela sigla WORA (Write Once Run Anywhere) que significa “Escreva uma vez, execute em qualquer lugar”.

JRE (Java Runtime Environment) é um conjunto de software que forma um ambiente para a execução de bytecodes feitos em Java. Ela é composta da JVM e bibliotecas adicionais. JDK (Java Development Kit) é um conjunto de programas que todo programador Java deve ter. Ele é um kit de desenvolvimento Java composto pela JRE (item acima), o JavaLang, o JavaTools (JavaC, Debugger, APIs). Além disso, você ainda pode adicionar uma IDE para facilitar o desenvolvimento Java.

As IDEs mais conhecidas são o Eclipse, o IntelliJ e o NetBeans. O Curso em Vídeo de Java vai adotar o NetBeans para o aprendizado da Linguagem Java.

0:11 / 19:

A PROGRAMAÇÃO ORIENTADA A OBJETOS

Moreira Neto (2009) nos diz que o modelo orientado a objetos usa como suporte a execução de métodos (pequenas funções que agem normalmente sobre os dados de um objeto), dando importância à maneira como o usuário enxerga o sistema e suas funcionalidades. A programação orientada a objetos (POO) é o paradigma de programação que atualmente domina o mundo da programação, tendo substituído, conforme nos evidenciam Horstmann e Cornell (2010), as técnicas da programação estruturada que foram criadas nos anos 1970. A linguagem Java é totalmente orientada a objetos, mas o desenvolvedor para utilizá-la precisa estar totalmente habituado a POO.

Por causa das diferenças estruturais e conceituais, o modelo orientado a objetos é mais natural (mais próximo de nossa compreensão da realidade), e mais econômico no sentido de que necessita de menos passos para atingir um mesmo resultado. (MOREIRA NETO, 2009, p.8). Melo (2010) nos diz que em meados dos anos 1970, os métodos orientados a objetos começaram a aparecer reforçando ainda mais a mudança de paradigma.

A programação orientada a objetos já estava amadurecendo, bastava então criar para ela um caminho a se seguir e então deu início à análise orientada a objetos. O grande ganho com esse novo paradigma é o fato de que se padronizaram os modelos usados para análise, projeto e implementação. Ainda os principais modelos são usados em todas as fases, mudando-se apenas a visão.

Deitel e Deitel (2010) explicam que, em 1980, as organizações começaram a utilizar a Orientação a Objetos para criar aplicativos e desenvolver a OOAD (ObjectOriented Analysis and Design) padrão, onde muitos metodologistas as produziram com processos separados para essas necessidades, onde cada processo possuía sua própria notação para transportar os resultados de análises e design.

Assim, podendo modelar o software utilizando semelhanças descrevendo objetos da realidade aperfeiçoando o desenvolvimento do protótipo.

Ainda, segundo Deitel e Deitel (2010), a programação orientada a objetos introduz a característica de herança que é como se reutiliza um software de uma nova classe para aprimorá-lo com uma classe existente e com isso economiza-se tempo para o desenvolvimento de novas classes e aumenta a probabilidade de um sistema ser implementado e mantido efetivamente.

NetBeans IDE é um ambiente de desenvolvimento - uma ferramenta para programadores escrever, compilar, depurar e implantar programas. É escrito em Java - mas pode suportar qualquer linguagem de programação. Existe também um enorme número de módulos para aprimorar o NetBeans IDE. O NetBeans IDE é um produto gratuito sem restrições de como ser utilizado (NETBEANS, 2013).

Para Paula Filho (2009), na engenharia de software, os processos podem ser definidos para tarefas como desenvolvimento, manutenção, aquisição e contratação de software. Em um processo de desenvolvimento de sistemas, o ponto inicial para a arquitetura deste processo é a escolha de modelo de ciclo de vida.

Paula Filho (2009) ainda evidencia que no modelo de ciclo de vida em Cascata, os principais subprocessos são executados em rígida sequência, por isso é possível demarcá-los com pontos de controle muito bem definidos. Esses pontos de controle facilitam bastante a gerência dos projetos, o que faz com que esse processo seja, em princípio, confiável e utilizável em projetos de qualquer tamanho. “O modelo cascata pode ser muito útil para ajudar desenvolvedores a descrever o que eles precisam fazer. Sua simplicidade o torna fácil de explicar aos clientes não familiarizados como desenvolvimento de software” (PFLEEGER, 2004, p. 39)

Segundo Eckstein (2007), o Model, View, Controller (MVC), foi utilizado pela primeira vez por Trygve Reenskaug, um desenvolvedor do Centro de Pesquisas da Xerox em Palo Alto (EUA), em 1979, e tem como objetivo ajudar a separar o acesso a dados e regras de negócio da maneira em que é exibida para o usuário.

Mais precisamente, o MVC pode ser dividido em três elementos:

• Model (modelo) - O modelo representa os dados e as regras que governam o acesso e atualização de dados. Em uma empresa, um modelo muitas vezes serve como uma aproximação de um processo do mundo real software.

• View (visualização) - A visão retorna o conteúdo de um modelo. Ele especifica exatamente como os dados do modelo devem ser apresentados. Se há alterações de dados no modelo, a visão deve atualizar sua apresentação, se necessário.

• Controller (controlador) - O controlador traduz as interações do usuário com a visão em ações que o modelo irá executar. A figura abaixo demonstra que a entrada do usuário, a modelagem do mundo externo e o feedback visual para o usuário são separados e gerenciados pelos objetos Modelo (Model), Visão (View) e controlador (Controller).

Segue abaixo o modelo MVC

Figura 1: modelo MVC

Fonte: https://semanaacademica.org.br/system/files/artigos/uso_da_tecnologia_java_no_desenvolvimento_de_um_software_para_controle_de_producao.pdf

King (2011, et al) nos diz que o trabalho com software objeto relacional e banco de dados relacional, pode ser desagradável e cansativo nos dias de hoje em uma empresa e que o Hibernate é um Objeto/Relacional de Mapeamento de ferramentas nos meios Java.

O Hibernate não cuida apenas do mapeamento das classes de Java até as tabelas de banco de dados, mas também assegura, conforme salienta King (2011, et al) a consulta de dados e simplicidade para recuperação que pode influenciar e muito na redução do tempo de desenvolvimento. A meta de Hibernate é aliviar o desenvolvedor em 95% de dados comuns de persistência relacionados às tarefas de programação.

O Hibernate talvez não seja a melhor solução para as aplicações centradas em dados, das quais apenas usam procedimentos armazenados para a implementação das lógicas comerciais no banco de dados. Isto é mais utilizado orientando o objeto aos modelos de domínio e lógicas comerciais na camada intermediária baseada em Java. (KING et al., 2011, p.11).

Para Elmasri e Navathe (2011), os bancos e suas tecnologias, têm um peso importante sobre o aumento do uso dos computadores. E é certo afirmar que os bancos de dados realizam um papel essencial em quase todas as áreas em que os computadores são utilizados, incluindo negócios, comércio eletrônico, engenharia, medicina, genética, direito, educação, biblioteconomia. O MySQL atualmente é o mais popular banco de dados livre do mundo e é desenvolvido, distribuído e suportado pela Oracle Corporation.

O MySQL oferece um rápido e robusto servidor de banco de dados SQL (Structured Query Language) multithreading e multi-usuário. (MySQL Team, 2013) Para King (2011, et al), no Hibernate também é possível realizar consultas utilizando o dialeto SQL nativo de seu banco de dados, pois ele permite que que o programador especifique o SQL escrito à mão, incluindo procedimentos armazenados, para todas as operações como criar, atualizar, deletar e carregar.

Ela também oferece um caminho de migração limpo de uma aplicação em SQL/JDBC direta até o Hibernate. Rodrigues Filho (2007) nos diz que o JDBC é a sigla que representa: Java Database Connectivity e é a biblioteca de classes de Java que dá acesso ao banco de dados. Ela tem um conjunto de interfaces que criam um ambiente comum em que as aplicações e os mecanismos de banco de dados se comunicam.

Ainda Elmasri e Navathe (2011) nos dizem que o SQL, é uma linguagem de banco de dados imensa: tem comandos para definição de banco de dados, consultas e atualizações. “SQL é uma linguagem de definição e de manipulação de dados relacionais, desenvolvida nos laboratórios da IBM (International Business Machines) nos anos 70 e hoje padronizada pelos comitês ISO/Ansi”. (GUIMARÃES, 2003, p. 99) Oliveira (2002) nos evidencia que a linguagem SQL não é uma linguagem de programação independente, ela poderia ser chamada de sub-linguagem, pois quando se desenvolvem aplicações para banco de dados, é preciso utilizar uma linguagem de programação tradicional e utilizar comandos SQL para manipular os dados.

Moreira Neto (2009) afirma que, geralmente, os dados usados e as informações que resultam dos processamentos devem ser salvos em um repositório, que pode ser um arquivo de texto, um arquivo binário, ou, de forma mais tecnológica, um banco de dados relacional. O banco é usado em 95% dos sistemas comerciais por disponibilizar repositórios de dados seguros e robustos.

Segundo Benthin (2010), o MySQL Workbench é uma ferramenta gráfica para modelagem, integrando, criação, designer e administração de banco de dados MySQL. A ferramenta possibilita trabalhar diretamente com objetos schema, além de fazer a separação do modelo lógico do catálogo de banco de dados. Combina características profissionais e uma interface clara e simples para oferecer a forma mais eficiente de lidar com as suas bases de dados.

O JAVA

Java é uma linguagem de programação 100% orientada a objetos, que foi lançada em 1995 pela Sun Microsystems, porém segundo o DEITEL (2003), a criação do Java se iniciou em 1991 inicialmente com o nome de Green. Esta linguagem é base para praticamente todo tipo de aplicativos de rede.

O foco inicial era criar uma tecnologia de desenvolvimento de software para aparelhos domésticos (Atualmente podemos ver Java em vários eletrodomésticos, sendo também base para a tv digital brasileira).

Seu criador, James Gosling, o batizou com o nome de Oak, que era uma árvore que ele sempre observava, porém, depois de muito estudo, foi descoberto que já existia uma linguagem chamada de Oak.

Segundo Pacievitch, em 2006, o Java já tinha sido quase todo convertido para a licença de Software livre, criando assim uma comunidade de desenvolvimento global. Sua utilização é relativamente simples, por isso a quantidade de programadores que o adotam aumenta a cada dia (PACIEVITCH, 2011).

Com a popularização da internet, a equipe de desenvolvedores, começou a ter a ideia de implantá-lo na internet. Em seu lançamento em 1995, foi liberada a primeira versão do java, a qual já suportava desenvolvimento para a internet. Com sua enorme aceitação, grandes empresas como a IBM, anunciaram que ajudariam no suporte ao Java. Com isso, as empresas fariam seus produtos aceitarem aplicativos feitos em Java. Segundo estudos, Java foi a tecnologia adotada mais rapidamente na história da informática.

Esta linguagem passou a ser utilizada em vários dispositivos como os computadores, palmtops, celulares e em grande parte dos produtos da Apple.

A linguagem Java oferece mecanismos de encapsulamento, abstração e hereditariedade; é independente de plataforma, possui uma máquina virtual a JVM (Java Virtual Machine), que transforma o código do programa em bytecodes, linguagem própria da JVM; é uma linguagem compilada e interpretada não vinculada com ambiente de desenvolvimento, a sua restrição é apenas ter o JDK que é composto pelo compilador, máquina virtual e as bibliotecas de desenvolvimento instalado no computador (JUNIOR, 1999).

Para escrevermos um programa desenvolvido em Pseudocódigo de forma que o computador possa compila-lo e executá-lo, precisamos utilizar uma Linguagem de Programação que nada mais é do que o “idioma” necessário para conversar com o computador. No desenvolvimento de nossos estudos em Lógica de Programação utilizaremos a linguagem de programação Java.

O Java atualmente é uma das linguagens de programação mais utilizadas no Mercado de Trabalho, ele também é capaz de fornecer uma portabilidade muito grande, sendo compatível com a maioria dos Sistemas Operacionais disponíveis para usuários gerais. O mesmo programa escrito com a Linguagem de Programação Java, poderá ser utilizado em um computador com os Sistemas Operacionais Windows, Linux ou Mac OS sem que nenhuma linha do código fonte necessite ser alterada, ganhando tempo no desenvolvimento de novas aplicações.

Além disto, o Java é uma linguagem de programação que pode ser utilizada para desenvolver páginas da Internet, através de um Servidor Web configurado para executar páginas do tipo Java Server Pages (jsp). Com ele também podemos desenvolver aplicativos para celulares que utilizam o Sistema Operacional Android.

Características da Linguagem Java

- **Parecida com C, C++:** a sintaxe da linguagem Java é muito semelhante a da linguagem C ou de C++.

Logo, o programador que já conhece a linguagem C achará a transição para Java simples e fácil.

A sintaxe de Java fornece uma versão mais limpa do que C++. Em Java não existe aritmética de ponteiros, estruturas, uniões e etc.

- **Orientada a Objetos:** Java é uma linguagem voltada para a programação orientada a objetos e, por isso, todo o código está contido dentro de classes.

Java suporta herança simples, mas não herança múltipla. A ausência de herança múltipla pode ser compensada pelo uso de herança e interfaces, onde uma classe herda o comportamento de sua superclasse além de oferecer uma implementação para uma ou mais interfaces.

- **Compilada e Interpretada:** Todo programa Java é compilado e interpretado. Um programa em Java é compilado para um código composto por instruções chamadas de “bytecode”. O “bytecode” é um código de uma máquina virtual, chamada Máquina Virtual Java (Java Virtual Machine - JVM), idealizada pelos criadores da linguagem. Os bytecodes são independentes de plataforma e são interpretados pela JVM para serem executados no computador.

Todo o interpretador Java ou browser que execute applets Java é uma implementação de uma Máquina Virtual Java. A JVM também pode ser implementada em hardware.

Além da JVM, a plataforma Java é composta também pela Java Application Programming Interface (Java API). A API Java é uma grande coleção de componentes de software disponibilizados que fornecem muitas capacidades interessantes e úteis, tais como, componentes de interface gráfica, conexão via sockets, etc. A API Java é agrupada em bibliotecas (packages ou pacotes) de componentes relacionados.

O código Java é compilado uma única vez, porém o programa é interpretado toda vez que for executado. Esse processo pode ser mais bem visualizado na figura a seguir.

ONDE O JAVA ESTÁ PRESENTE

O Java é rápido, seguro e confiável. De laptops a datacenters, consoles de games a supercomputadores científicos, telefones celulares à Internet, o Java está em todos os lugares! A linguagem Java é uma das mais difundidas e conta com inúmeros recursos pela Web. Além disso há padrões de TV digital que disponibilizam algum suporte a Java. Sendo assim, Java está disponível para um amplo espectro de aplicações, cada qual com sua utilidade e características próprias. Só para ter uma ideia da dimensão da utilização, atualmente mais de 3 bilhões de dispositivos executam Java.

Java é uma das linguagens de programação mais populares usadas para criar aplicações e plataformas Web. Foi desenhado para flexibilidade, permitindo que os programadores escrevam códigos que seriam executados em qualquer máquina, independentemente da arquitetura ou plataforma. De acordo com a página inicial do Java, mais de 1 bilhão de computadores e 3 bilhões de telemóveis no mundo executam Java.

Java é usado para construir aplicações e plataformas para vários dispositivos, incluindo computadores, laptops, consolas, players de Blu-ray, sistemas de navegação para carros, dispositivos de monitoramento médico, parquímetros, terminais de loteria e smartphones. É também uma linguagem chave para redes, particularmente para data centers que armazenam e transferem dados baseados na Web.

Há muitos lugares onde o Java é usado no mundo real, a partir de um site de comércio eletrônico comercial para aplicativos Android, de aplicações científicas a aplicações financeiras como sistemas de negociação eletrónicos, de jogos como o Minecraft a aplicativos de desktop como o Eclipse, NetBeans e IntelliJ. , de uma biblioteca de código aberto para aplicativos J2ME, etc. Vamos ver cada um deles em mais detalhes:

Aplicações Android

Nunca se está muito longe de ver onde Java é usado no mundo real. Se abrir qualquer aplicação no seu telemóvel Android, as aplicações estão escritas em Java, com a API do Google Android, que é semelhante ao JDK. À cerca de dois anos atrás, o Android ajudou a impulsionar esta necessecidade e hoje muitos programadores de Java desenvolvem aplicações Android.

Aplicações de servidor na indústria de serviços financeiros

Java é muito vasto em serviços financeiros. Muitos bancos de investimento globais como o Goldman Sachs, o Citigroup, o Barclays, Standard Charted e outros bancos usam o Java para escrever sistemas de negociação eletrónica de frente e verso, sistemas de liquidação e confirmação, projetos de processamento de dados e vários outros. Java é usado principalmente para escrever aplicações do lado do servidor, principalmente sem qualquer front end, que recebe dados de um servidor (upstream), processa e envia outro processo (downstream).

Aplicativos de Web Java

O Java também é grande no comércio eletrónico e no espaço de aplicações da web. Você tem muitos serviços RESTfull sendo criados usando Spring MVC, Struts 2.0 e frameworks semelhantes. Até mesmo simples aplicativos da Web baseados em Servlet, JSP e Struts são bastante populares em vários projetos do governo. Muitos dos departamentos de governo, saúde, seguros, educação, defesa e vários outros departamentos têm seu aplicativo da Web construído em Java.

Ferramentas de Software

Muitas ferramentas úteis de software e desenvolvimento são escritas e desenvolvidas em Java, e. Eclipse, InetelliJ Idea e Netbans IDE. Hoje em dia, Java FX está a ganhar popularidade, mas ainda não é um substituto do Swing e C #.

Aplicação de Negociação

Negociação de terceiros, também faz parte da maior indústria de serviços financeiros, também usa Java. Um aplicativo de negociação popular como o Murex,

usado em muitos bancos para conectividade de bancos, também é escrito em Java.

Aplicativos J2ME

Embora o advento do iOS e Android quase descartou o J2ME, ainda há um grande mercado de low-end Nokia e Samsung aparelhos que usa J2ME. Houve uma altura em que quase todos os jogos e aplicações, que estão disponíveis no Android eram escritos em MIDP e CLDC, parte da plataforma J2ME. J2ME ainda é popular em produtos como Blu-ray, cartões, Set top boxes etc. Uma das razões do WhatsApp ser tão popular é porque também está disponível no J2ME para todos os aparelhos Nokia.

Espaço Embutido

Java também é vasto no que toca a espaço embutido. Mostra o quão capaz é a plataforma, apenas são precisos de 130 KB para poder usar a tecnologia Java (num cartão inteligente ou sensor). Originalmente, o Java foi projetado para dispositivos embarcados. Na verdade, essa é a única área, que fazia parte da campanha inicial de Java "escrever uma vez, trabalhar em qualquer lugar" e parece que está a colher frutos agora.

Tecnologias de Big Data

O Hadoop e outras tecnologias de big data também usam Java de uma forma ou de outra, e. HBase e Accumulo (open source) baseados em Java do Apache e ElasticSearch também. Por Java já não estar a dominar esse campo, existem tecnologias como o MongoDB que está escrito em C ++. Java tem potencial para obter grande participação neste espaço crescente se o Hadoop ou o ElasticSearch crescerem.

Espaço de Negociação de Alta Frequência

A plataforma Java melhorou muito as suas características de desempenho e, com as modernas JITs, é capaz de fornecer desempenho no nível C ++. Devido a esse motivo, o Java também é popular na criação de sistemas de alto desempenho, porque o desempenho é um pouco menor em comparação com o idioma

nativo, mas a segurança pode ficar comprometida, a portabilidade e a capacidade de manutenção para obter mais velocidade e leva apenas um programador C ++ inexperiente para fazer um aplicativo lento e não confiável.

Aplicações Científicas

Atualmente, o Java é frequentemente uma opção padrão para aplicativos científicos, incluindo o processamento de linguagem natural. A principal razão disso é que o Java é mais seguro, portátil, sustentável e vem com melhores ferramentas de simultaneidade de alto nível do que o C ++ ou qualquer outra linguagem.

AS PRINCIPAIS VANTAGENS DO JAVA

Para esta linguagem ser tão contada certamente existem vários motivos. Vamos então, agora, relacionar as principais vantagens do JAVA:

- Java tem a filosofia ‘Escreva uma Vez e Execute em Qualquer Lugar’

Com isso você poupa tempo em aprender bibliotecas específicas. Poupa esforço, pois muitas vezes não há a necessidade de se conhecer alguns ou vários aspectos da plataforma.

- Java possui “Coleta de Lixo” automática.

Isso significa que ela desaloca automaticamente qualquer espaço de memória não utilizado sem que o programador se preocupe com isso.

- Java tem alta produtividade

Por ser uma linguagem de alto nível Java tem construtores e características que permitem escrever programas mais rapidamente do que em C/C++. Sendo assim há uma maior produtividade, sendo uma ótima escolha para desenvolvimento.

- Java está presente em uma variedade de lugares

Os aplicativos Java estão presentes em diversas áreas, desde celulares, até servidores, o que dá ao desenvolvedor um vasto leque de possibilidades de sistemas a serem desenvolvidos. Mais de 3 bilhões de dispositivos executam o Java.

- Java tem uma boa documentação, é gratuito e de código aberto

Como não há custo para se desenvolver em Java, pois a linguagem é gratuita e de código aberto, é um incentivo para quem quer aprender a programar pois não necessitam gastar com ferramentas pagas, muito pelo contrário existem escolhas de altíssima qualidade gratuitas e amplamente usadas.

PLATAFORMAS JAVA

Como já é de conhecimento existem diferentes ferramentas para desenvolvimento que o Java oferece, que serão relembradas abaixo:

JSE: Java Standard Edition. A versão básica do Java com ela é possível desenvolver diversos aplicativos para desktop, normalmente aqueles que querem se aprofundar nessa tecnologia começam por essa plataforma. Existem pacotes para se desenvolver programas em console ou, por exemplo, Swing que se pode desenvolver softwares com interface gráfica.

JME: Java Micro Edition é a plataforma voltada para o desenvolvimento de embarcados e dispositivos móveis, atualmente essa plataforma é muito popular entre os celulares. Ela foi dividida em duas frentes: Para os dispositivos com poder computacional limitado CLDC, Connected Limited Device Configuration, e dispositivos com um poder computacional um pouco maior CDC, Connected Device Configuration.

Java TV: Dentro da plataforma JME, o Java TV traz uma API que ter permite o desenvolvimento para a Televisão Digital com recursos como fluxo de áudio e vídeo, acesso aos dados nos canais de transmissão, sincronia das mídias dentre outros. Esta tecnologia veio a ser amplamente estudada e desenvolvida após a definição do padrão de televisão digital brasileiro.

Java FX: Essa plataforma visa a criação da Internet rica, aplicações web com características de um programa tradicional de um desktop, em diversos dispositivos (desktop, browser, telefone celulares, TVs, videogames, Blu-ray players etc.).

JEE: Java Enterprise Edition é uma versão voltada para as empresas e para aplicativos web. Uma grande característica dessa plataforma é a necessidade de um servidor de aplicação. Recentemente recebeu a atualização para a versão 6, dentre os frameworks dessa plataforma podemos citar:

JSP (Java Server Pages): De uma maneira geral ele executa Java em sua página.

JPA (Java Persistence API): Ele funciona como um padrão dos padrões de persistência. Com ele se pode modificar de framework de persistência (hibernate, toplink etc) com impacto zero em sua aplicação.

JSF (Java Server Faces): Framework responsável pela camada de visão muito indicado para programas que utilizarão o padrão MVC (Model View Control), uma de suas características é o uso do AJAX de maneira nativa.

Java Card: Programação voltada para o desenvolvimento de cartões inteligentes. Um dos seus principais benefícios do seu uso é a portabilidade e a segurança. Esses dispositivos vêm crescendo nos últimos anos e tendem a crescer ainda mais.

Java Virtual Machine – JVM

O que faz com que a portabilidade do Java seja eficiente é uma aplicação responsável por executar programas desenvolvidos na linguagem. Sua função é simular um computador permitindo a execução do código fonte, por isto recebe o nome de Máquina Virtual. Na prática, basta instalar em seu computador a JVM desenvolvida para o Sistema Operacional que você estará pronto para executar programas desenvolvidos em Java.

O Java Devlopment Kit (JDK) e a Aplication Programming Interface (API) Eclipse

Como em breve seremos desenvolvedores e não usuários, também precisaremos instalar em nosso computador o kit para desenvolvimento de programas feitos em Java, o que inclui o compilador4 da linguagem de programação e a Máquina Virtual (JVM). Sem ele não é possível finalizar um programa desenvolvido em Java, mesmo que você escreva o código fonte completo. Apesar de já ser possível criar programas apenas com o JDK, utilizaremos uma interface de

desenvolvimento (API) para nos auxiliar na escrita, compilação e testes dos nossos programas. O Java tem como principais APIs de desenvolvimento o NetBeans e o Eclipse. Independente da API escolhida os comandos sempre serão os mesmos, logo, se você aprende a programar em Java, conseguirá utilizar qualquer uma das APIs sem maiores problemas. Durante o desenvolvimento das atividades utilizaremos a API Eclipse, porém você pode utilizar qualquer outra API desde que siga fielmente as estruturas listadas neste material.

VÍDEO DE APOIO

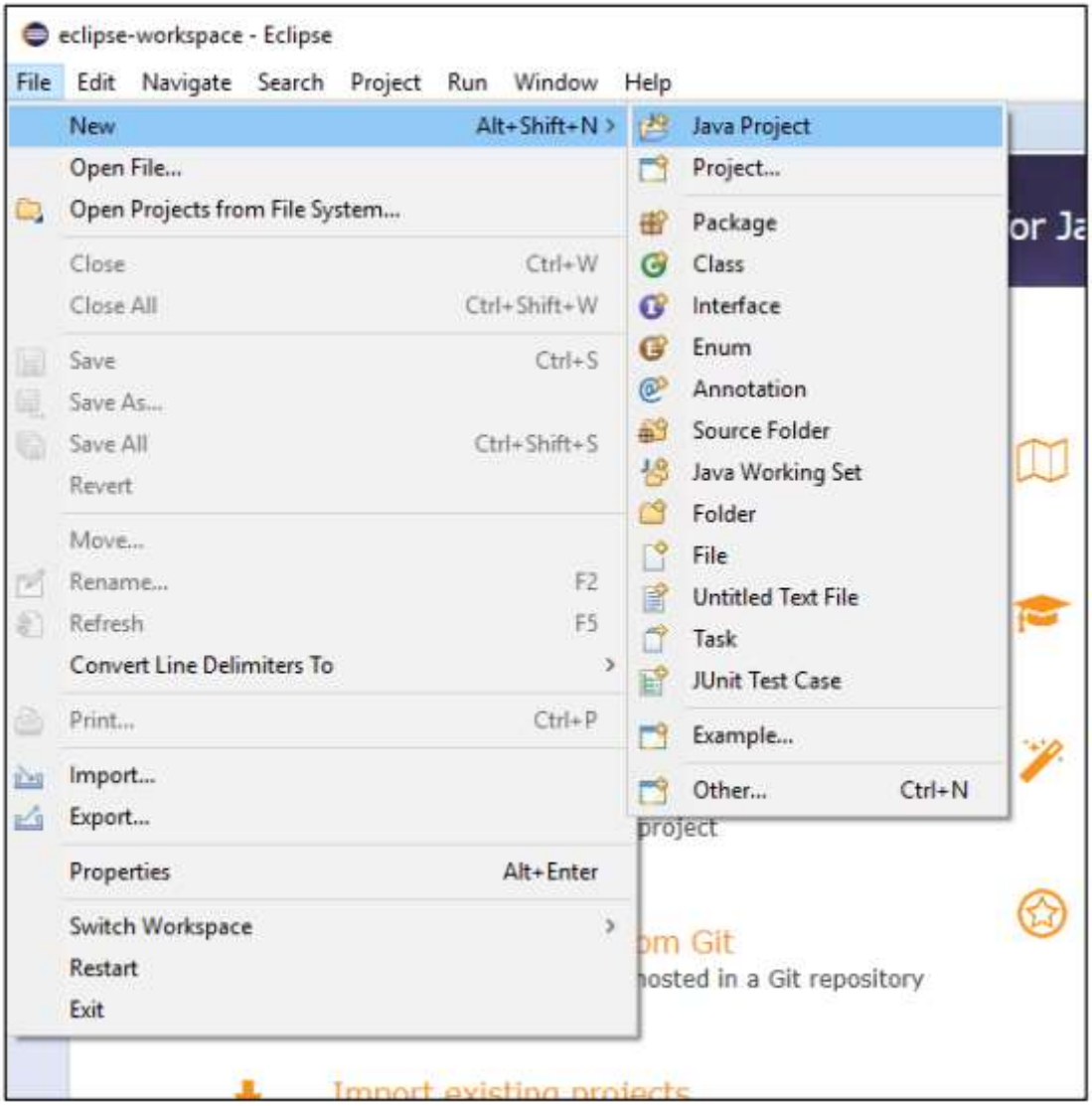
Vídeo 2: Criando primeiro programa em Java - Curso de Java - Aula 01
Disponível em:< https://www.youtube.com/watch?v=X8AnVQ-GqLU>
Sinopse: neste vídeo do canal CFBCursos o prof. Bruno, ensina as noções básicas de Java, a configurar o ambiente para criar os programas Java e também criar um programa em Java.
Java é uma das linguagens de programação mais usadas no mundo, uma linguagem Orientada a objetos que tem uma portabilidade muito grande, o código escrito serve para qualquer Sistema Operacional que tenha a JVM instalada.

Criando um projeto utilizando a API Eclipse

Na estrutura de organização do Eclipse, cada programa desenvolvido é caracterizado como um Projeto. E cada código fonte, será tratado como uma Classe. Como utilizaremos o Java como uma Linguagem de Programação estruturada, não entraremos a fundo no significado de Projeto, Pacote e Classe, pois estas definições são utilizadas em programas Orientados a Objetos.

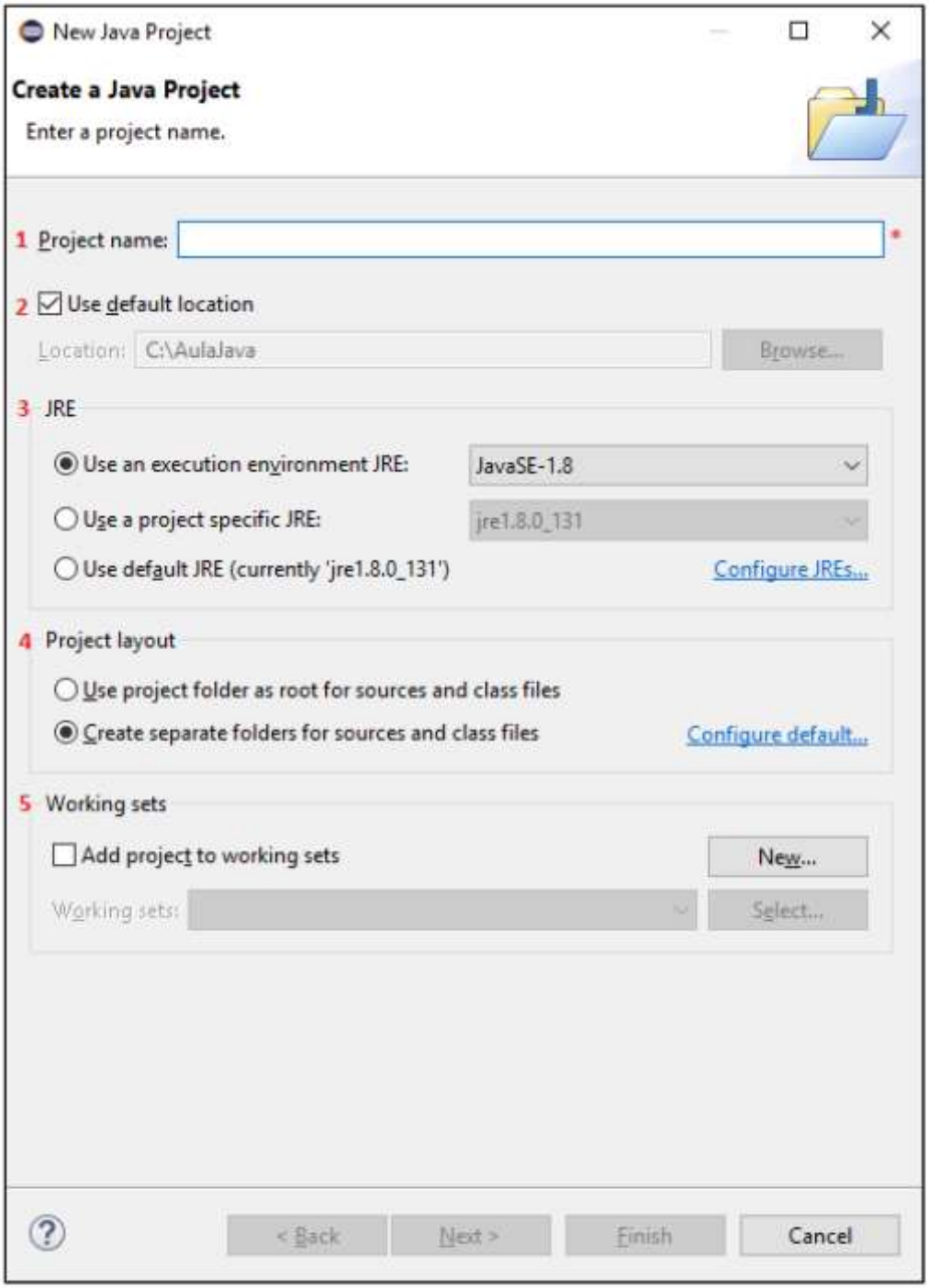
Para criar um novo programa, após abrir a janela principal do Eclipse, seleciona-se o menu File, New, Java Project, conforme a figura 2 a seguir:

Figura 2: Passo a passo – janela para criar programa



A seguinte janela será exibida:

Figura 3: Passo a passo – janela para criar programa



As opções fornecidas são:

1. Nome do Projeto: nesta opção você deve indicar um nome para o seu programa. O nome do programa deve ser simples e objetivo. Não é recomendado que seja utilizado espaços para separar palavras. Caso seja necessário, utilize o underline. Este campo é obrigatório.
2. Selecionar onde ficará salvo o projeto: é altamente recomendável que todos os projetos desenvolvidos com o Eclipse sejam salvos em um único lugar, preferencialmente no local indicado ao iniciar o Eclipse. Desmarque a opção “Use default location” apenas se a sua intenção for salvar o seu projeto em um local diferente do padrão.
3. JRE: você só deve alterar esta opção caso você tenha o JDK instalado em seu computador em mais de uma versão e deseje testar seus programas em uma versão mais antiga do JDK.
4. Project Layout: nesta opção você indicará como o Eclipse deve organizar os arquivos de seu programa. O ideal é manter a opção padrão selecionada, pois ela manterá seus códigos fontes sempre em uma pasta separada do restante do projeto.
5. Working Sets: apenas para usuários avançados. Com esta opção você poderá mesclar dados do seu projeto com o de outros projetos localizados em diferentes workspaces.

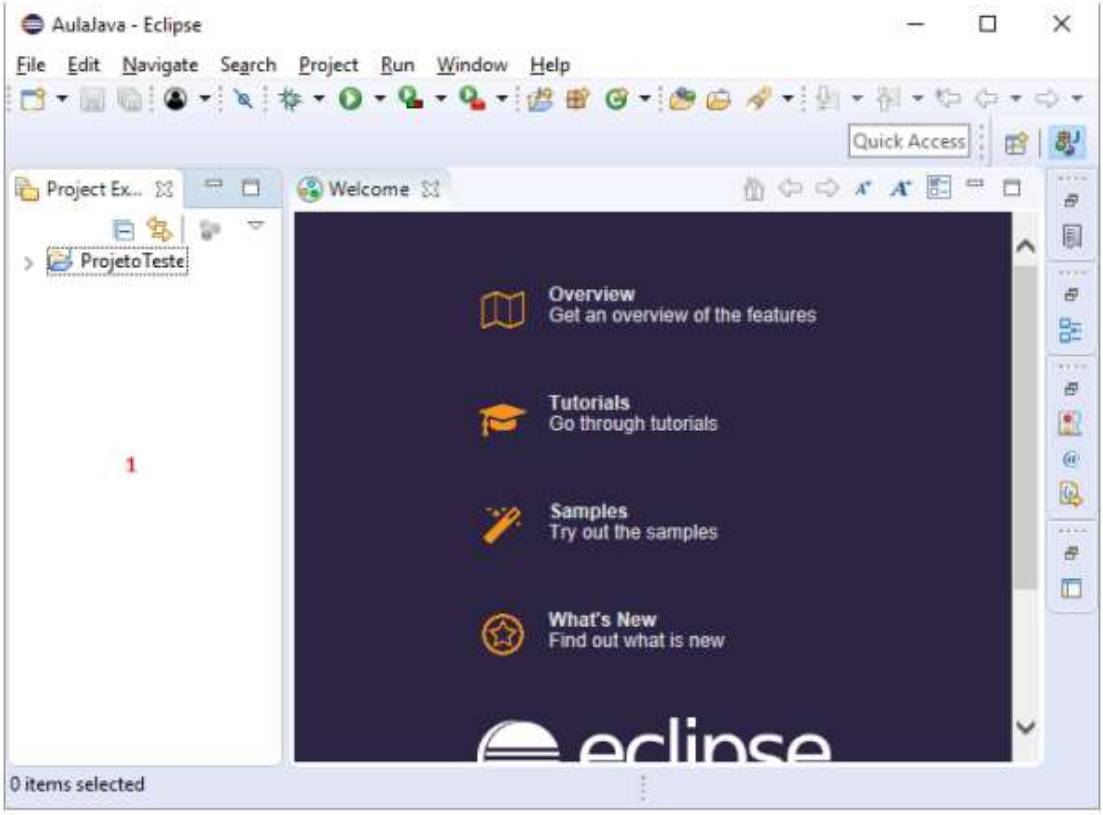
Após inserir um nome para o seu projeto e alterar as demais opções, caso seja necessário, clique em Finish. E voltará para a tela inicial, porém desta vez haverá um pequeno botão ao lado esquerdo ou direito da do Eclipse, dependendo da versão utilizada. O botão aparenta conforme a imagem a seguir:

o clicar no botão,

Figura 4: Arquivo GEEaD

aparecerá a nossa interface do projeto conforme a imagem a seguir:

Figura 5



Note que ao lado direito, marcado com o número 1, temos a janela Project Explorer. Será por esta Janela que localizaremos os arquivos contidos no nosso projeto.

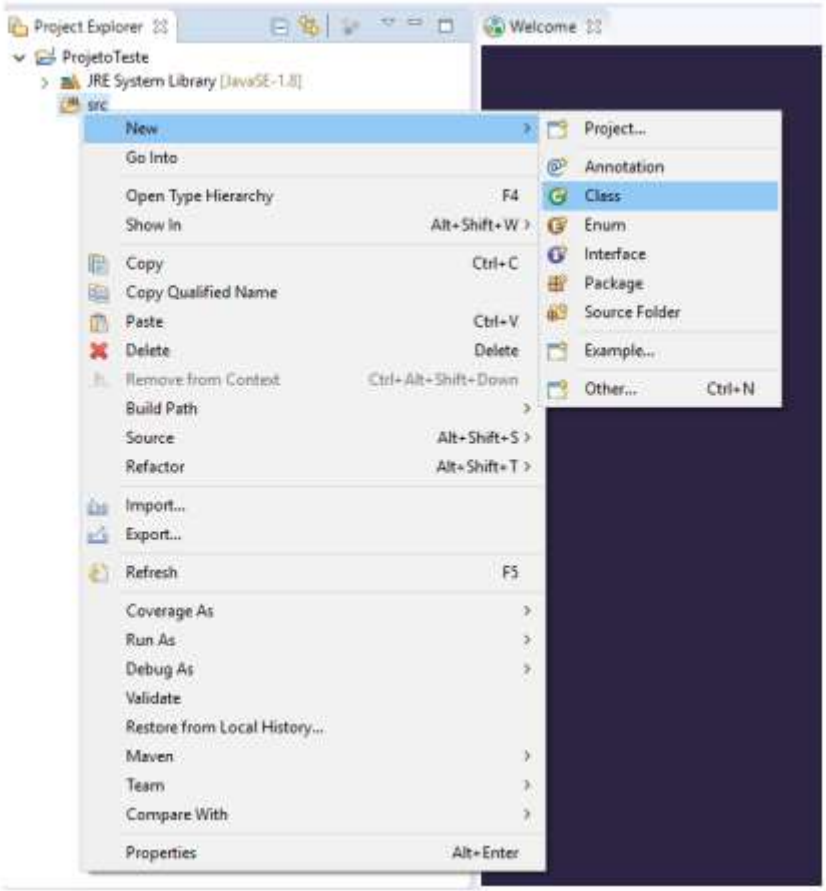
Importante: Caso você feche acidentalmente a janela Project Explorer, poderá acessá-la novamente através do menu Windows > Show View > Project Explorer.

Agora que temos o projeto, vamos criar um arquivo para o nosso código fonte:

Em Project Explorer, entre na pasta referente ao seu projeto, localize a pasta chamada src , clique com o botão direito nela e selecione New > Class:

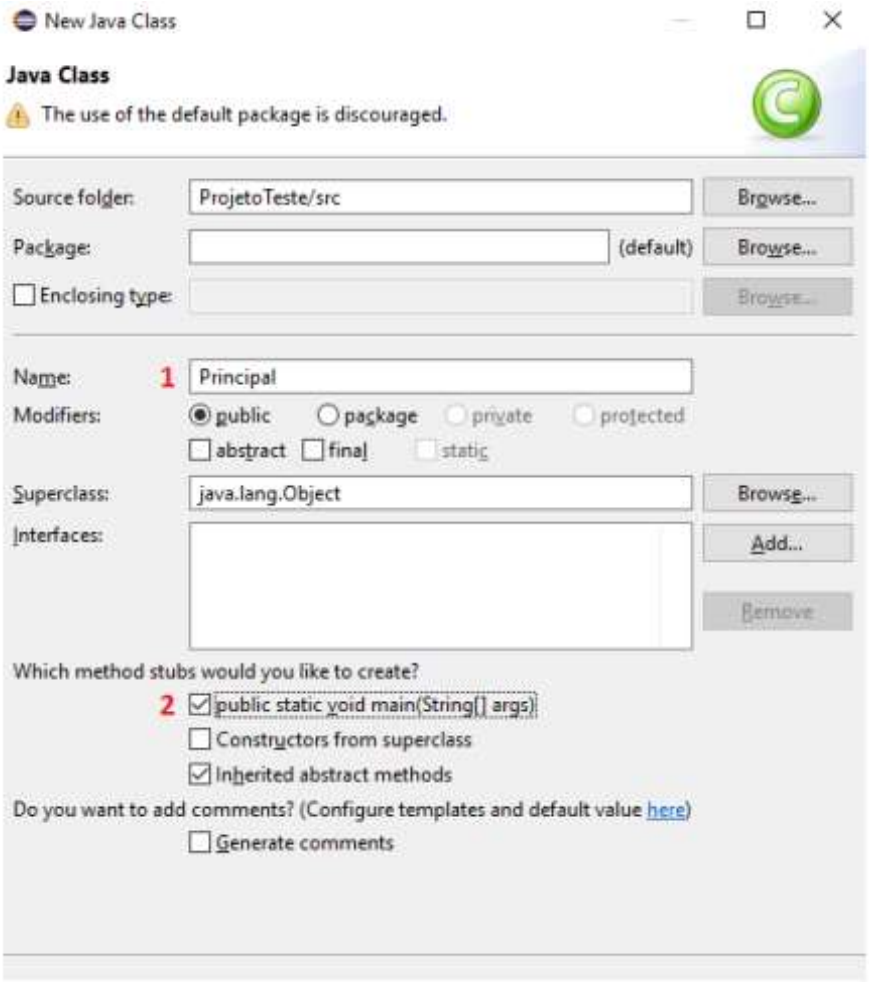
Relembrando: a pasta src (abreviação para source Code) é a pasta padrão do eclipse para receber códigos fontes

Figura 6



Por não utilizarmos a Orientação a Objetos na programação Java, precisaremos apenas definir um nome para o arquivo (1) e selecionar a opção “public static void main(String[] args)” (2) :

Figura 7



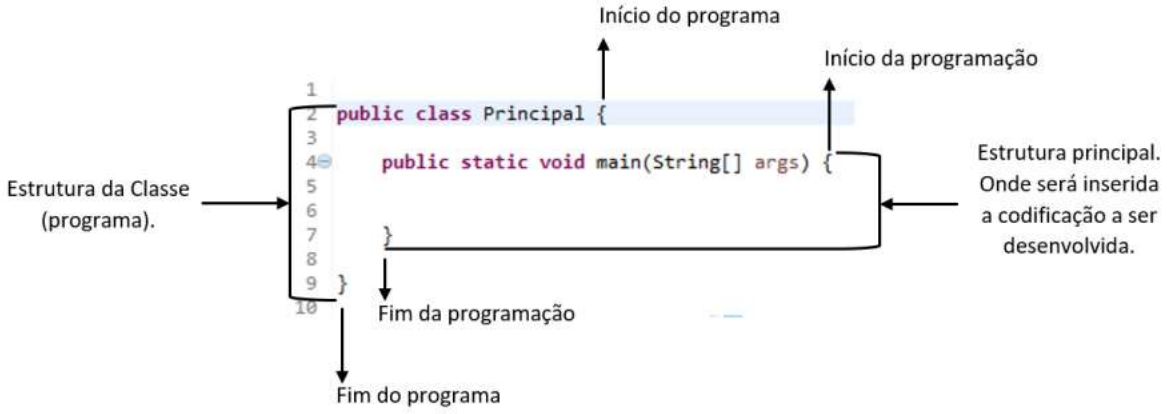
LEMBRE-SE: assim como na criação do projeto, o nome do arquivo de código fonte não pode conter espaços, acentuação e caracteres especiais.

A estrutura de um programa feito em Java

Agora que já foi criado um código Fonte em Java, será necessário aprender a estrutura básica de um programa.

A seguir temos uma imagem representando um código fonte em Java:

Figura 8



Pode-se ver na imagem acima que o programa em si está dentro de uma Classe. Esta estrutura é necessária para permitir que no futuro o seu programa em Java se torne um programa Orientado a Objetos.

Os comandos Início e Fim contidos no Pseudocódigo, são substituídos por chaves { } em Java, sendo a abertura de Chave { o início e, o fechamento de chave } o final do seu programa. Em algumas estruturas que veremos posteriormente, também utilizaremos marcações de início e fim através de chaves. A partir de agora, os novos comandos apresentados a você sempre serão apresentados em Java, além da sua forma utilizada no Fluxograma e no Pseudocódigo.

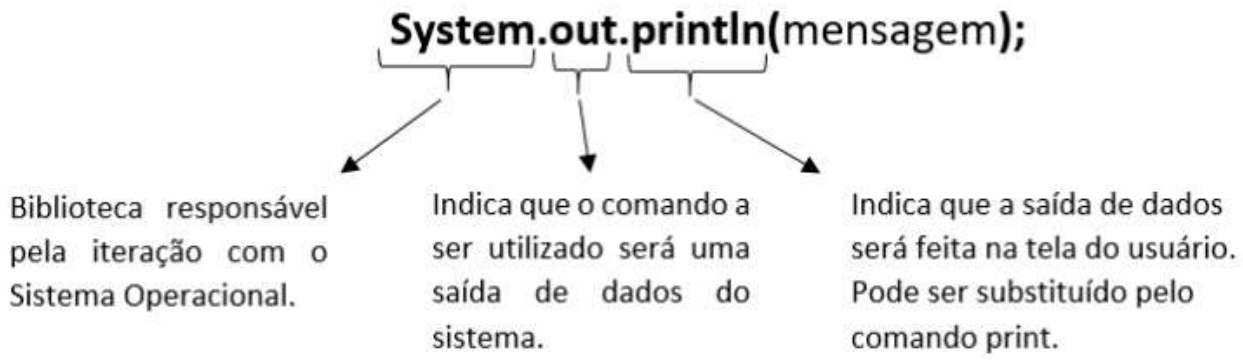
O comando Escreva

Quando desejamos exibir alguma mensagem na tela do usuário utilizando o pseudo-código, utilizamos o comando:

escreva *mensagem*

Com o auxílio dele é possível desde enviar uma simples mensagem ao usuário do programa, como também mostrar o resultado de uma conta. Na prática: tudo o que você deseja que o usuário veja, deverá utilizar este comando para mostrar. Em Java, por ser uma linguagem de programação, deve-se indicar o caminho completo da operação de exibição de mensagem para o usuário, que é feita da seguinte forma:

Figura 9



O comando print escreverá a mensagem na tela sem efetuar nenhuma modificação na mensagem.

O comando println pulará uma linha na tela, antes de iniciar a escrita da mensagem.

Portanto, se a intenção é exibir a mensagem grudada na mensagem exibida anteriormente, deve-se utilizar o print, caso deseje que a mensagem seja exibida em uma nova linha, utilize o println.

Por exemplo: Desejamos que o usuário veja a seguinte mensagem:

Olá, você está bem?

Figura 10

Fluxograma	Pseudocódigo	Java
"Olá, você está bem?"	escreva "Olá, você está bem?"	System.out.println("Olá, você está bem?");

Declarando uma variável

Figura 11

Tipos de variáveis Pseudocódigo	Tipos de variáveis Java	Valores compreendidos dentro do tipo	Exemplo de valores em Java.
Inteiro	byte	Números entre -128 e 127.	10
	short	Números entre -32768 e 32767.	13320
	Int	Números entre -2147483648 e 2147483647.	-170000000
	long	Números entre -9223372036854775808 e 9223372036854775807.	14000222999333
Real	float	Números reais entre -10^{38} até 10^{38}.	0.134
	double	Números reais entre -10^{308} até 10^{308}.	143.4938293019283
Caractere	char	Um único caractere (letra), entre apóstrofos. Exemplo: 'a'.	'd'
	String	Mais de um caractere, entre aspas. Exemplo: "Técnico em Informática".	"Informática"
Lógico	boolean	Verdadeiro ou Falso.	falso

Note que em Java, o único tipo de dado que se inicia com letra maiúscula é o String. Vale lembrar também que os números decimais são separados por ponto (.) ao invés de vírgula. Voltando ao caso da Agenda Telefônica, qual seria o tipo de dados que o número de telefone dado pelo Juvenal seria?

O número de telefone seria uma variável do tipo String, pois apesar de ser um número de telefone, não efetuamos nenhum tipo de cálculo com este número, sendo assim, não é necessário definir este tipo de dado como inteiro.

Após identificar qual será o tipo da variável que você utilizará, basta declará-la em seu programa, seguindo o padrão para cada linguagem de programação, sendo:

Figura 12

PSEUDOCÓDIGO	JAVA
declare	
nomedavariavel *como* ***tipododado***	***tipododado*** **nomedavariavel;**

Abaixo, temos um exemplo de sua aplicação utilizando o fluxograma, pseudocódigo e o Java:

	Fluxograma	Pseudocódigo	Java
Exemplo	nota1 como real nota2 como real nota3 como real media como real	declare nome como caractere idade como inteiro preco como real	String nome; Int idade; Double preco;

As variáveis devem ser nomeadas de forma objetiva, que esclareça facilmente o programador qual é a sua função, para garantir um rápido entendimento e continuidade do desenvolvimento do software caso necessário.
Quando nomeamos as variáveis, é imprescindível também seguir algumas regras, que são:

1. As variáveis nunca podem conter um espaço em seu nome.

Figura 13

nome do aluno como caractere ✗	nomeAluno como caractere ✓
String data nascimento; ✗	String data_nascimento; ✓

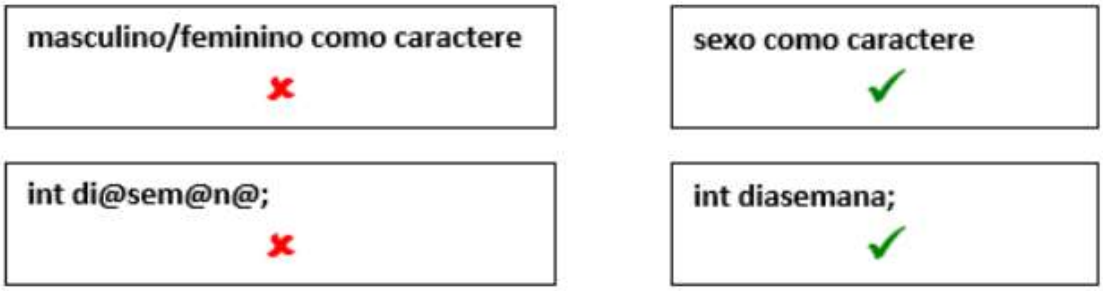
Caso você necessite de mais de uma palavra para definir o nome de uma variável, junte as palavras ou então separe-as apenas com um **underline** _ .

Remova as preposições do nome da variável, assim o nome dela ficará mais objetivo e fácil de entender. Não inclua mais do que duas palavras em um nome de variável, seja sempre objetivo.

2. As variáveis nunca podem conter caracteres especiais em seu nome.

Em uma linguagem de programação, os caracteres especiais são palavras reservadas que são utilizadas pela linguagem para trabalhar comandos especiais, cálculos etc. Se você utilizar caracteres especiais em nome de variáveis, o computador não entenderá se ele deverá demarcar o espaço da variável na memória ou se iniciará algum procedimento especial do computador, por isto tentar colocar um nome destes resultará em erro de compilação* .

Figura 14

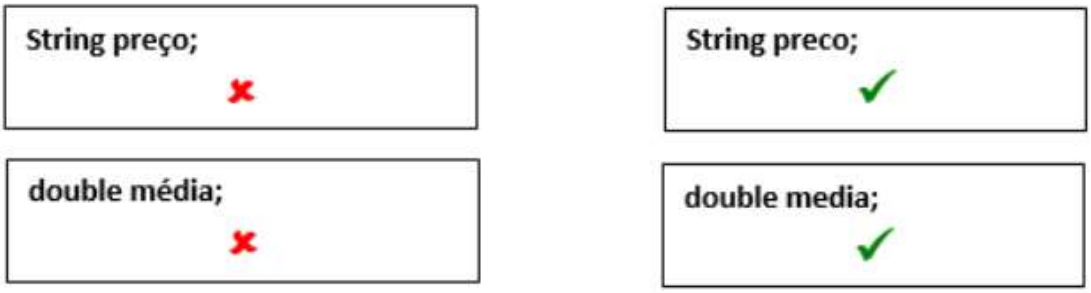


* Erros de compilação são configurados quando há erros na estrutura do código fonte. Caso este tipo de erro ocorra, não será possível a execução do programa.

Entende-se por caracteres especiais os seguintes sinais: !, @, #, $, %, \ , /,], [, (,), {, }, e todos os caracteres não alfanuméricos.

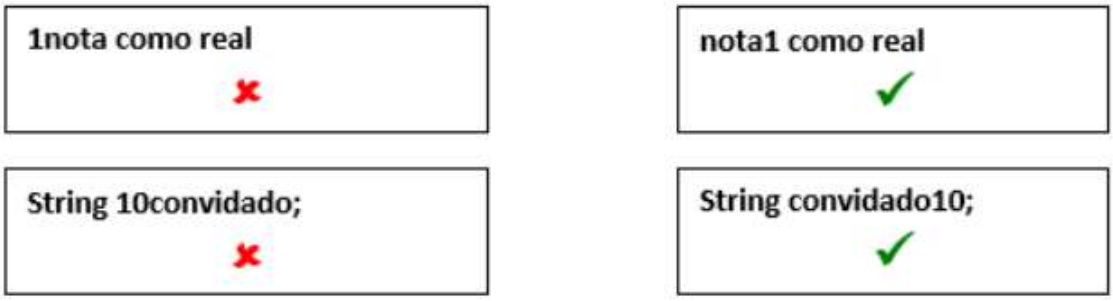
2. Nomes de variáveis não podem receber acentuação. Como as linguagens de programação são todas escritas no idioma Inglês, onde não há acentuação nas palavras, não podemos utiliza-los em declarações de variáveis. O cêcedilha (Ç) apesar de ser um caractere, também entra nesta regra.

Figura 15



4. Nomes de variáveis não podem ser iniciados por números. Quando a linguagem de programação encontra um número em uma codificação, logo ela entende que deverá ser feito um cálculo. Caso este número venha junto com um caractere em seguida, o computador não identificará o caractere como sendo um número válido para efetuar um cálculo, assim gerando um erro de compilação. Você poderá utilizar um número normalmente no nome da sua variável desde que este número não seja o primeiro caractere de seu nome.

Figura 16



O comando Leia

O comando Leia é o comando responsável por receber dados inseridos pelo usuário. Em geral estes dados são inseridos através do teclado, podendo ser numérico ou caractere dependendo do tipo de dados que a variável que receberá o valor estiver configurada. Como o computador não saberá qual será o valor que o usuário digitará, sempre teremos que utilizar uma variável para armazenar o valor obtido através do comando Leia.

A sintaxe do comando Leia é:

Figura 17

Pseudocódigo:

leia(variável)

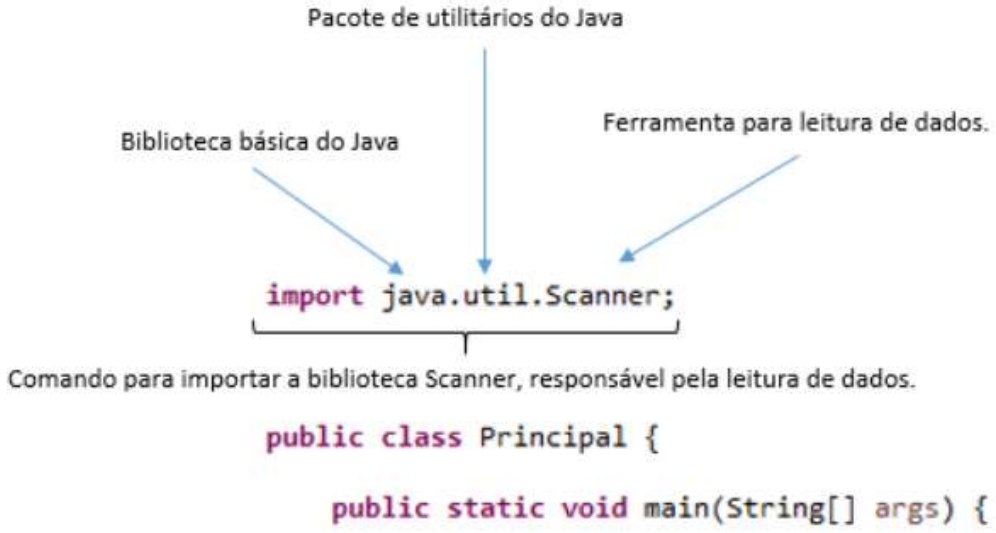
Variável que receberá o dado digitado pelo usuário.

Java

Em Java um programa inicial contém apenas o suporte à exibição de mensagens no monitor, processamentos básicos de dados e utilização de variáveis com o objetivo de garantir a otimização do espaço de seu programa em disco e consequentemente, o peso da sua aplicação. Para que seja possível a utilização de recursos diferentes, é necessário realizar uma importação de uma biblioteca de classes para o seu Projeto.

Estas bibliotecas contém as instruções necessárias para que o Java consiga trabalhar com novas funções conforme a necessidade do programador. Vale lembrar que não é recomendado que você faça uma importação de biblioteca em seu programa caso não seja a intenção utiliza-la, podendo acarretar perca de desempenho desnecessária na sua aplicação. Para importar uma biblioteca, basta seguir o seguinte comando:

Figura 18



Note que o comando import deve ser inserido na primeira linha do seu código, antes mesmo de todos os códigos gerados automaticamente pela IDE Eclipse.

IMPORTANTE:

O Java possui inúmeras bibliotecas que poderão ser importadas sempre que necessário. Além disto também pode-se importar bibliotecas feitas por outras pessoas, com o objetivo de poupar muito trabalho no desenvolvimento de uma nova ferramenta da estaca zero.

Após a importação da ferramenta Scanner, precisamos "cria-la" dentro do nosso programa, utilizando o seguinte comando:

Figura 19

O comando para criar o leitor dentro do nosso programa é chamado de instância. Na instância é onde a sua ferramenta importada cria vida, tornando-se funcional e utilizável na sua aplicação. A partir deste momento, o leitor será carregado na memória do computador junto com a sua aplicação.

Leitura de dados utilizando a ferramenta Scanner

Agora que já se tem o leitor, estamos prontos para ler uma entrada de dados feita pelo usuário através do teclado e armazena-la em uma variável. Para que esta leitura seja feita de forma adequada pelo Java, devemos adotar uma leitura específica para cada tipo de variável, conforme a tabela a seguir:

Figura 20

Tipo da variável que receberá o dado (Java)	Comando utilizado pelo leitor	Exemplo
byte	leitor.nextByte();	byte numero; numero = leitor.nextByte();
short	leitor.nextShort();	short numero; numero = leitor.nextShort();
int	leitor.nextInt();	Int numero; numero = leitor.nextint();
long	leitor.nextLong();	long numero; numero = leitor.nextLong();
float	leitor.nextFloat();	float numero; numero = leitor.nextFloat();
double	leitor.nextDouble();	double numero; numero = leitor.nextDouble();
char	leitor.next().charAt(0);	char letra; letra = leitor.next().charAt(0);
String	leitor.next();	String palavra; palavra = leitor.next();
boolean	leitor.nextBoolean();	Boolean teste;
		teste = leitor.nextBoolean();

Exemplo prático de um programa

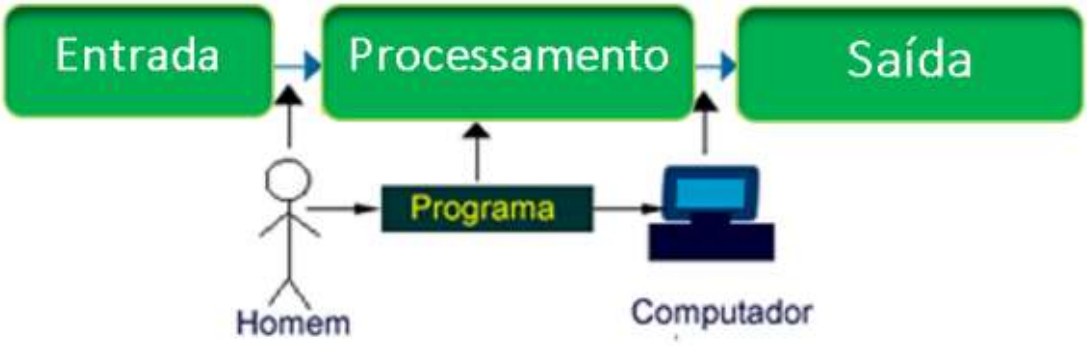
Passo 1: Definir a sequência lógica do programa.

Antes de construir o fluxograma, será necessário identificar a entrada, o processamento e a saída dos dados dentro do nosso programa:

Entrada: o programa deverá solicitar para que o usuário digite dois números, do tipo numérico.

Processamento: o programa calculará a soma destes números. Saída: O programa exibirá a soma destes números.

Figura 21



Passo 2: Definir quais serão as variáveis necessárias para o programa.

Seguindo a definição de variáveis, precisa-se identificar quais dados não serão fixos no programa. Verificando as informações construídas no passo 1, nota-se que os dois números que o usuário digitar e a soma deles não são fixos, podendo ser modificado a cada vez em que o usuário utilizar o programa.

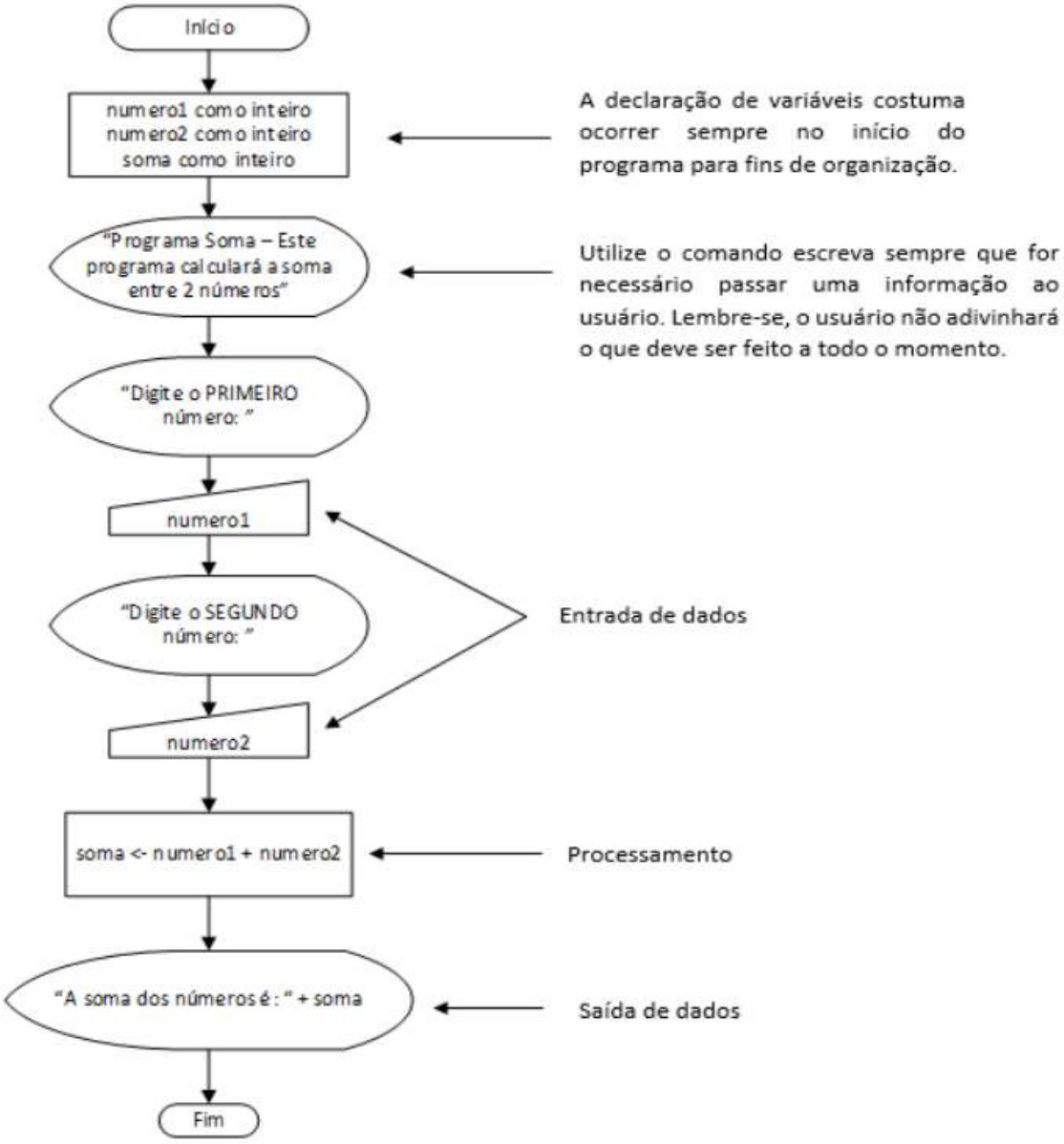
Logo, as variáveis serão:

numero1 como inteiro
numero2 como inteiro
Soma como inteiro

Passo 3: Construir o Fluxograma.

Agora que já se sabe as variáveis e a sequência lógica do programa, pode-se construir o fluxograma, conforme a simbologia apresentada anteriormente:

Figura 22



LEMBRE-SE: sempre procure identificar a entrada, o processamento e a saída dos seus dados em qualquer fluxograma, assim a chance de conter erros no fluxograma cairá consideravelmente.

Passo 4: Construir o Pseudocódigo

Após construir o fluxograma, ficará muito fácil criar o Pseudocódigo, basta "traduzir" a simbologia do Fluxograma para o Pseudocódigo:

Declare

Numero 1 como inteiro

Numero 2 como inteiro

Soma como inteiro

Inicio

Escreva("Programa Soma – Este programa calculará a soma entre 2 números")

Escreva("Digite o PRIMEIRO número")

Leia(numero1)

Escreva("Digite o SEGUNDO número")

Leia(numero2)

Entrada

Soma <- numero1 + numero2

Processamento

Escreva("A soma dos números é: " + soma)

Fim

Saída

IMPORTANTE: Com o Pseudocódigo já é possível testar o nosso código, com o auxílio da ferramenta VisualG, caso você tenha interesse em testar o pseudocódigo execute-o utilizando a ferramenta.

Passo 5: Construir o programa em Java.

Assim como foi feito com o Pseudocódigo, basta aplicar as regras básicas da linguagem Java. Em primeiro lugar, será criado um novo projeto para este exemplo da mesma forma que foi visto anteriormente.

A seguir, será aplicado o Pseudocódigo, adaptando-o para a linguagem Java. Não pode esquecer de importar a biblioteca responsável pela leitura de dados:

Figura 23

```
*SomaValores.java
1  import java.util.Scanner;
2  public class SomaValores {
3
4      public static void main(String[] args) {
5          /*  O código desenvolvido por você deverá estar aqui.
6              As chaves marcam o inicio do código
7              Diferente do pseudocódigo, as variáveis em java podem ser declaradas diretamente
8              no corpo do programa, mas a recomendação continua: devemos declarar as variáveis
9              logo no incio do código fonte.
10         */
11         //declaração das variáveis
12         int numero1;
13         int numero2;
14         int soma;
15         //para facilitar o entendimento, também habilitamos o leitor no inicio do código.
16         Scanner leitor = new Scanner(System.in);
17         //inicio do programa
18         System.out.println("Programa Soma - Este programa calculará a soma entre 2 números");
19         System.out.println("Digite o PRIMEIRO valor");
20         //leitura do primeiro valor
21         numero1 = leitor.nextInt();
22         System.out.println("Digite o SEGUNDO valor");
23         //leitura do segundo valor
24         numero2 = leitor.nextInt();
25         //processamento
26         soma = numero1 + numero2;
27         //saída de dados
28         System.out.println("O resultado da soma é "+ soma);
29     }
30 }
```

Algumas considerações:

1. há explicações precedidas por duas barras // ou /* ?Estes sinais indicam comentários do Java, com eles você poderá escrever qualquer mensagem para o programador ou então anotar informações importantes sobre o seu código. Estes códigos não serão processados pelo computador.

   a. Os comandos para iniciar um comentário em Java são:

   b. // comentário de uma única linha

   c. /* i. Comentário de múltiplas linhas

   d. */

   e. A codificação indicada como comentário ficará por padrão na cor verde.

2. O Java é uma linguagem Case Sensitive, ou seja, o Java diferencia as letras maiúsculas de minúsculas. Para o Java a letra “a” minúscula é diferente da letra “A” maiúscula. Por este motivo, os comandos da

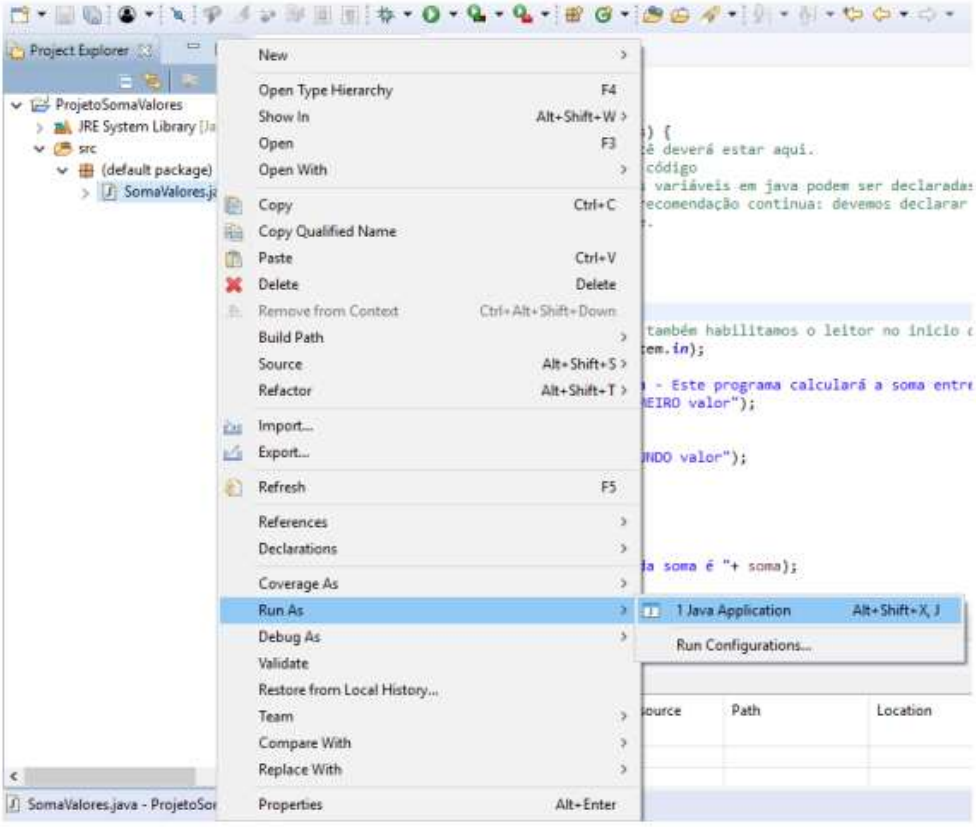
linguagem devem ser reproduzidos fielmente a sua apresentação, caso contrário o comando não será reconhecido. É o caso do System.out.println(), onde o S deve ser sempre digitado em letra maiúscula.

3. Todo o comando feito em Java deve ser finalizado por um ponto e vírgula (;). É através do ponto e vírgula que o Java entende que o comando foi finalizado, executando-o e preparando-se para receber o próximo comando. Caso você esqueça do ponto e vírgula (que por sinal é o erro mais comum), o seu programa não funcionará.
4. Caso o seu texto digitado com auxílio do Eclipse fique sublinhado em vermelho, significa que ele está com erro. Caso isto ocorra, primeiro você deve sempre corrigi-lo, caso contrário seu programa não funcionará. Geralmente estes erros são causados por “erro de sintaxe”, em outras palavras, o comando foi digitado incorretamente.
5. Comandos sublinhados em amarelo não são erros. Geralmente são recomendações ou aviso de variáveis declaradas, mas não utilizadas. 6. Se você passar o mouse em cima de palavras sublinhadas, a IDE apontará as recomendações necessárias para corrigir o problema. Mas atenção: nem sempre a IDE acertará o palpite, lembre-se que o computador não é perfeito, portanto o ideal é sempre rever o código e entender o que está errado.

Passo 6: Executando o programa.

A última etapa do nosso programa é executa-lo e testar para ver se tudo ocorreu bem. Para testá-lo, basta clicar no nome da sua classe com o botão direito (em Project Explorer) e selecionar a opção Run > Java Application:

Figura 24



Abaixo do Código Fonte, verá uma janela chamada Console, onde poderá inserir os dados para a utilização de seu programa:

Figura 25

```
SomaValores.java
1  import java.util.Scanner;
2  public class SomaValores {
3
4      public static void main(String[] args) {
5          /*  O código desenvolvido por você deverá estar aqui.
6              As chaves marcam o inicio do código
7              Diferente do pseudocódigo, as variáveis em java podem ser declaradas diretamente
8              no corpo do programa, mas a recomendação continua: devemos declarar as variáveis
9              logo no incio do código fonte.
10         */
11         //declaração das variáveis
12         int numero1;
13         int numero2;
14         int soma;
15         //para facilitar o entendimento, também habilitamos o leitor no inicio do código.
16         Scanner leitor = new Scanner(System.in);
17         //inicio do programa
18         System.out.println("Programa Soma - Este programa calculará a soma entre 2 números");
19         System.out.println("Digite o PRIMEIRO valor");

Problems  Javadoc  Declaration  Console
<terminated> SomaValores [Java Application] C:\Program Files\Java\jre1.8.0_131\bin\javaw.exe (19 de set de 2017 21:14:10)
Programa Soma - Este programa calculará a soma entre 2 números
Digite o PRIMEIRO valor
3
Digite o SEGUNDO valor
2
O resultado da soma é 5
```

Caso a janela Console não apareça, pode-se abri-la acessando o menu Window > Show View > Console.

IMPORTANTE: vale lembrar que caso haja erros em seu Código Fonte, o Eclipse retornará uma mensagem de erro, não sendo possível executar o seu código enquanto os erros (sublinhados em vermelho) não sejam corrigidos.

Aprimorando a comunicação com o usuário

Antes de mais nada, os programas devem ser fáceis de se utilizar e os usuários devem fazer isso intuitivamente. Para tanto todo programador deve atentar-se a interface com o usuário. Vamos aprender uma maneira alternativa de entrada de saída de dados mais elegante para utilizarmos em nossos futuros programas em Java. Até aqui utilizamos o que chamamos de modo console, ou seja, uma tela de terminal que não aceitava elementos gráficos para interagirmos com o programa.

Figura 26

```
Problems   @ Javadoc   Declaration   Console
<terminated> ApresentadoConsol [Java Application] C:\Progran
Entre com um nome:
 José
O nome é:  José
```

Observação

Um programador deve sempre tentar desenvolver uma interface com o usuário que seja simples de ser utilizada. Contudo essa interface deve agradar ao cliente que está contratando os seus serviços. Pensando como o programador, reflita como isso pode interferir na prática de programação antes de prosseguir a leitura.

Atualmente vivemos em um mundo onde os elementos gráficos predominam em todos os aspectos de comunicação. Portanto será apresentado um modo gráfico utilizando as janelas do sistema operacional para realizarmos a entrada e a saída de dados.

Entrada de dados

Para realizarmos a entrada de dados para um programa utilizaremos o comando JOptionPane do Java.

A sintaxe do comando é a seguinte:

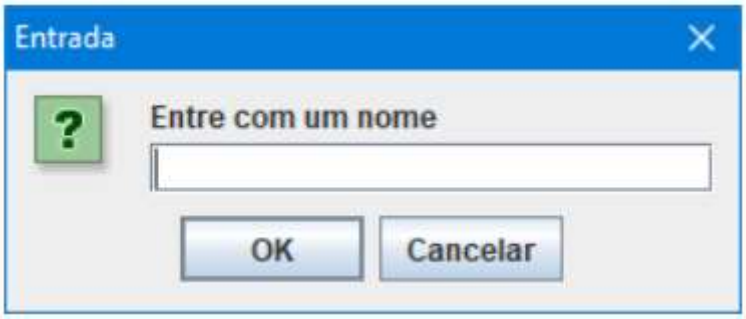
Figura 27

Figura 28

```
*JoptionPane.java
1  import javax.swing.JOptionPane;
2
3  public class JoptionPane {
4
5      public static void main(String[] args) {
6
7          String entrada; // variável de entrada
8          entrada = JOptionPane.showInputDialog("Entre com um nome");
9
10     }
11
12 }
13
```

Na linha 7 realizamos a declaração da variável entrada do tipo String que armazenará o conteúdo inserido pelo usuário. Na linha 8 a variável entrada recebe o conteúdo do comando **JOptionPane.showInputDialog**(“Entre com um nome”);. Vamos analisar como esta linha se comporta. Este comando solicita ao usuário que digite algum dado para o sistema. Mas qual dado é este? No nosso exemplo o dado solicitado é o nome – Entre com um nome. O resultado para o usuário será:

Figura 29



Um resultado muito mais bonito esteticamente que o modo console.

Saída de dados

Para realizarmos a saída dos dados de um programa do Java também utilizaremos o comando **JOptionPane**, mas com a seguinte sintaxe:

Figura 30

Figura 7 - Arquivo GEEaD

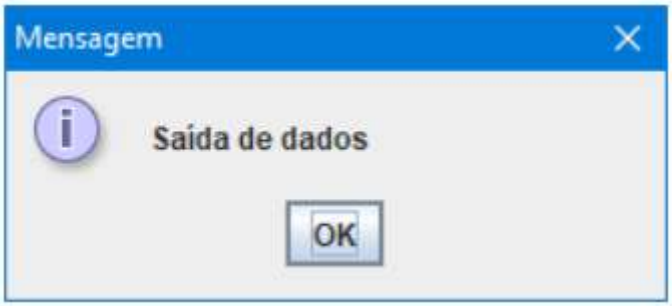
Exemplo de aplicação em um programa:

Figura 31

```
JoptionPaneShow.java
1  import javax.swing.JOptionPane;
2
3  public class JoptionPaneShow {
4
5      public static void main(String[] args) {
6          JOptionPane.showMessageDialog(null, "Saída de dados");
7
8      }
9
10 }
11
```

Na linha 6 o comando JOptionPane.showMessageDialog(null, “saída de dados”); faz a saída da mensagem para o usuário. O primeiro argumento sempre será null, seguido pela mensagem que queremos exibir para o usuário – Saída de dados. O resultado será:

Figura 32



Agora sabendo como realizar a entrada e a saída de dados de forma gráfica, conseguimos fazer um programa que leia e exiba dados para o usuário em modo gráfico, como no exemplo a seguir:

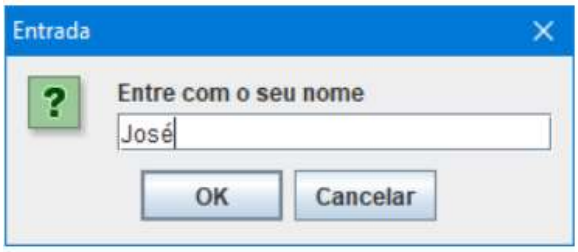
Figura 33

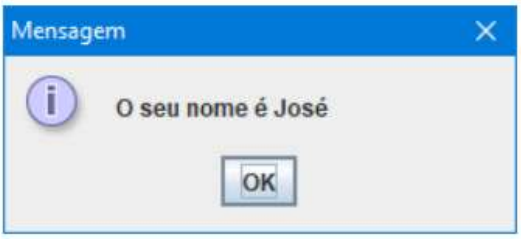
```
JOptionPane_E_S.java
1  import javax.swing.JOptionPane;
2
3  public class JOptionPane_E_S {
4
5      public static void main(String[] args) {
6
7          String nome; //Variável nome do tipo String
8
9          //entrada de dados
10         nome = JOptionPane.showInputDialog("Entre com o seu nome");
11
12         //saída de dados
13         JOptionPane.showMessageDialog(null, "O seu nome é " + nome);
14     }
15
16 }
17
```

Figura 8 - Arquivo GEEaD

Neste exemplo apresentado o programa exibe uma janela pedindo o nome do usuário na linha 10 e posteriormente exibe o nome digitado na linha 13. Resultado:

Figura 34





Conversão de tipos

O comando JOptionPane.showInputDialog sempre gera uma saída de dados do tipo String (sequência de caracteres alfanuméricos). Portanto se utilizarmos tipos de variáveis diferentes como, por exemplo, inteiro, devemos fazer a conversão de tipos antes de armazenarmos na sua variável correspondente conforme o exemplo a seguir:

Figura 35

JoptionPaneCast.java

```
1  import javax.swing.JOptionPane;
2
3  public class JoptionPaneCast {
4
5      public static void main(String[] args) {
6          String auxiliar; // variável auxiliar
7          int numeroInteiro;
8          double numerodouble;
9          float numerofloat;
10
11         //entrada de dados salvando na variável auxiliar (String)
12         auxiliar = JOptionPane.showInputDialog("Entre com um número inteiro");
13
14         //conversão do tipo String para Inteiro - Integer.parseInt
15         numeroInteiro = Integer.parseInt(auxiliar);
16         numerodouble = Double.parseDouble(auxiliar);
17         numerofloat = Float.parseFloat(auxiliar);
18
19         //Mensagem de saída
20         JOptionPane.showMessageDialog(null, "O número inteiro é "+ numeroInteiro);
21         JOptionPane.showMessageDialog(null, "O número double é "+ numerodouble);
22         JOptionPane.showMessageDialog(null, "O número float é "+ numerofloat);
23     }
24
25 }
26
```

Um tipo nada mais é do que o conteúdo que uma variável consegue armazenar. Um tipo String pode armazenar caracteres e números. Um tipo int números inteiros e os tipos double e float números reais.

No exemplo da figura acima a entrada de dados feita na linha 12 é salva na variável auxiliar que é do tipo String. Porém estamos trabalhando com números, nesse exemplo, e um dado do tipo String armazena texto. Para isto devemos fazer a conversão de tipo (cast em inglês), ou seja, temos que realizar a conversão de um tipo de dado para outro. Ex: de String para int, de String para Double, etc.

O Java possui comandos específicos para executar o cast.

Figura 36

A sintaxe para a conversão de um tipo **String para Inteiro** é:

Integer.parseInt(variável);

Tipo de dado a ser convertido, nesse caso Inteiro

Nome da variável a ser convertida

A sintaxe para a conversão de um tipo **String para Double** é:

Double.parseDouble(variável);

Tipo de dado a ser convertido, nesse caso Double

Nome da variável a ser convertida

A sintaxe para a conversão de um tipo **String para Float** é:

Float.parseFloat(variável);

Tipo de dado a ser convertido, nesse caso Float

Nome da variável a ser convertida

Sabendo a sintaxe do comando, na linha 15 a variável numeroInteiro recebe o resultado da conversão do valor da variável auxiliar de String para Inteiro. Assim como as linhas 16 e 17 realizam o cast para double e float, respectivamente.

Um aspecto interessante na linguagem Java é que no comando JOption-Pane.showMessage Dialog, não precisamos realizar a conversão de tipos para o String. O Java automaticamente converte tipos numéricos para String antes de exibir a mensagem para o usuário.

Uma observação: existem outras formas de realizarmos a conversão de tipos e para mais tipos de dados, porém não serão apresentadas aqui nesse momento.

JSF

Na atualidade, existem muitos frameworks para desenvolvimento de interfaces gráficas para software web. O JSF é um framework baseado em componentes que permite a criação de interface. Por exemplo, se quiséssemos exibir uma tabela com linhas e colunas, não seria necessário gerar tags de HTML para linhas e células como em um loop, basta adicionar um componente de tabela na página.

Podemos dizer que JSF é baseado nas seguintes regras:

- uma série de componentes pré-fabricados para criação de UIs (User Inte-faces).
- Um modelo de programação baseado em eventos
- um modelo de componentes que permite qualquer usuário desenvolver seus próprios elementos adicionais.

JSF não é o único framework web baseado em componentes, mas é a camada de visão no padrão Java EE. JSF está incluído em cada servidor de aplicativos Java EE, e pode ser facilmente adicionado a um web container como o Tomcat.

PRIMEFACES

Para atender a demanda dos desenvolvedores por componentes mais sofisticados, há várias extensões do JSF que seguem o mesmo ciclo e modelo da especificação. Exemplos dessas bibliotecas são PrimeFaces, RichFaces e IceFaces. Todas elas definem componentes JSF que vão muito além da especificação. Cada biblioteca oferece ShowCases na web para mostrar seus componentes e suas funcionalidades.

Onde se pode ver o showcase do PrimeFaces no endereço http://www.primefaces.org. No seu demo online, se pode ver uma lista de componentes disponíveis, como inputs, painéis, botões diversos, menus, gráficos e componentes drag & drop, que vão muito além das especificações, ainda mantendo a facilidade de uso.

VÍDEO DE APOIO

Vídeo 3: Curso de Java #09 – Estruturas Condicionais (Parte1)”
Disponível em:< https://www.youtube.com/watch?v=wW3eve4vTMc>

Sinopse: Aula do Curso de JAVA criado pelo professor Gustavo Guanabara para o portal CursoemVideo.com.

OS PASSOS PARA A IMPLANTAÇÃO DE UM SISTEMA JAVA

Então vamos lá, primeira situação:

- IDE, ou ambiente de desenvolvimento. No Java temos um conflito bem grande a respeito de sua IDE, no qual a maioria dos desenvolvedores ficam em dúvida entre qual ferramenta utilizar. Fica a critério de o desenvolvedor escolher a IDE que mais lhe agrada.

Ambiente de desenvolvimento NetBeans

Fonte: https://www.devmedia.com.br/tres-duvidas-comuns-ao-iniciar-um-sistema-em-java/28007

Segunda situação:

- Levantamento de plugin's, um simples exemplo. No decorrer do programa, ao tentar instalar um plugin no qual será detalhado melhor depois. Temos a situação que se certas versões não serem compatíveis entre plugin e IDE pode acabar dando problemas sérios tendo até que desinstalar o NetBeans e os plug-ins na pior das hipóteses. Perdendo bastante tempo que poderia ser usado para um fim mais "lucrativo".

E a terceira e última situação:

- A pressa de terminar, entregar e claro receber... Então, como diz um velho ditado, a pressa é inimiga da perfeição, ou seja, a pressa é amiga do suporte!
- Agora que essas três situações foram passadas a vocês, segue a explanação mais detalhada de cada uma delas.

No primeiro caso, a escolha pelo ambiente de desenvolvimento vai de cada programador, escolha aquela que você conheça melhor, onde a adaptação é fácil. Segue algumas ferramentas free para quem não tem uma específica, ou pra quem não conhece nenhuma delas pode simplesmente baixar e testar, são elas: NetBeans, Eclipse e JBuilder.

Escolhida a ferramenta, vamos aos plug-ins.

Um bom exemplo de plug-ins e que praticamente todo o sistema deve conter são os Relatórios.

Cuidado ao escolher os plug-ins para o desenvolvimento verifique bem as compatibilidades, e o que você necessita. Caso a escolha dos plug-ins conflitar com a versão do ambiente de desenvolvimento, você pode ter incômodos de vários "níveis", pode ter apenas que abrir o IDE, e excluir os plug-ins manualmente ou na pior das hipóteses, o ambiente não abrir mais e o desenvolvedor ter que desinstalar tudo e instalar de novo para que volte a funcionar perfeito.

Um livro que serve de grande ajuda para quem está iniciando um projeto por iniciativa própria, é o livro "Programação Java para Web" dos autores Decio Heinzelmann Luckow e Alexandre Altair de Melo, nele é encontrado exemplos, e um tutorial de como proceder com os plug-ins desde o início. Dependendo do aplicativo usado para gerar o arquivo de relatório, uma boa ferramenta para desenvolvimento de relatórios é o IReport com o JasperViewer.

Este aplicativo é uma das ferramentas que mais agrada os desenvolvedores para criar os relatórios. Pois ele traz leiautes de projetos prontos, podendo só escolher ou mesmo começar de uma folha A4 em branco, inserir os dados que deseja que apareçam no relatório e dar um ok, talvez a distribuição dos campos em uma primeira formatação não seja a desejada, mas para alterar e deixar da maneira desejada é algo muito simples, fácil e rápido.

Uma dica que economiza bastante tempo, pesquisas na internet, e dores de cabeça, é: os plug-ins necessários para abrir o IReport pelo JasperViewer, estão dentro da pasta raiz do aplicativo IReport, basta adicionar os .jar necessários.

Antes de colocar a mão na massa e desenvolver, sempre é bom criar um padrão de desenvolvimento como fala a terceira situação, ou seja, padronizar as variáveis constantes e chamadas de método do sistema, algo que ao longo do desenvolvimento do sistema, pode se tornar um problema. Sem um padrão e documentação, após algum tempo o sistema se torna complicado de entender e ainda de fazer sua manutenção.

Na pressa, o desenvolvedor acaba passando por cima de alguns detalhes que podem se tornar bastantes importantes ao decorrer do sistema. Sempre é bom usar a “boa prática de programação”, pense que em algum momento, você precisará mexer em certa função, e quanto mais simples e limpa ela estiver, melhor será o entendimento fazendo assim, ser mais rápido o desenvolvimento. Rotinas com certo grau de lógica, sendo um tanto quanto complicado, é bom fazer comentários precisos, isso não significa dizer toda vez que uma variável int i, recebe valor x, mas sim, com palavras chaves, mostrar os pontos críticos da rotina, e explicar de maneira direta como ela funciona.

No caso das variáveis, temos várias dicas, de como proceder, que é o caso do PascalCase, ou do CamelCase, presente na maioria dos sistemas, mas o mais importante, é assumir um padrão que seja de fácil adaptação e entendimento posterior.

Certos métodos que serão utilizados variam vezes durante o programa, podem ser inseridos em Classes genéricas, nas quais você só utilizaria a chamada e economizaria bastante tempo e deixaria o código mais limpo.

Outro exemplo para um código limpo, no qual economiza trabalho e muito código, são as heranças, na quais as classes pai podem ter os métodos que as classes filhas utilizam, como por exemplo, um cálculo, ou algo do gênero.

O FUTURO DO JAVA

Assim como a tecnologia da informação, as linguagens passam por constantes mudanças para se adaptar aos novos mercados. Com o Java esse processo não é diferente. A seguir são mostradas algumas novidades que virão neste ou nos próximos anos.

Suporte a um maior número de Linguagem: O projeto “Da Vinci Machine” tem como maior objetivo incrementar ainda mais o número de linguagens que rodam em cima da JVM. A ideia é fazer, invokeDynamic, criar instruções na JVM para ativar bytecodes externos.

Java 7 e 8: Com o objetivo de trazer novidades mais rápido o Java 7 foi dividido em dois projetos (Java 7 e Java 8). O primeiro tem o objetivo de ser entregue no segundo semestre de 2011. Dentre seus recursos podemos destacar: switch com String, inicialização dinâmica de collection iniciado (diamante), tratar várias exceções em um mesmo bloco (multi-catch). O Java 8 está previsto para ser entregue no segundo semestre de 2012, e dentre as novidades podemos destacar: suporte a closures (projeto lambda), manipulação de I/O de maneira mais fácil (Automatic Resource Management).

Java FX 2.0: Previsto ainda para esse ano essa a remoção do Fx script para Java, isso permite o uso do javafx em linguagens alternativas como o Groovy e o Scala, além da integração com o HTML5. Essa versão tende a agradar a vários desenvolvedores da linguagem podendo ser visto como “Swing 2.0” sem falar que pode ser integrado com a web e para dispositivos móveis.

Java ME: Nessa plataforma promete ser incrementada a função de renderização de Javascript e HTML 5, com isso a plataforma ganha aplicações web além de se ganhar um browser baseado no webkit além da facilidade na inserção do javaFX 2.0 na plataforma mobile.

Java EE 7: Nesta plataforma voltada para aplicações web se trará as novas versões do JSF 2.1, JPA 2.1 e JAX-RS 2.0 a integração com o HTML 5 será muito forte, ferramentas para desenvolvimento em computação nas nuvens além de desta plataforma já está usando o potencial do Java SE 7.

TEXTO DE APOIO

Lógica de Programação

Autor: Paulo Sérgio de Moraes

Última Atualização: 04 de Abril de 2000

Unicamp - Centro de Computação - DSC

1 Introdução à Lógica de Programação

1.1 Lógica

A lógica de programação é necessária para pessoas que desejam trabalhar com desenvolvimento de sistemas e programas, ela permite definir a seqüência lógica para o desenvolvimento.

Então o que é lógica?

Lógica de programação é a técnica de encadear pensamentos para atingir determinado objetivo.

1.2 Seqüência Lógica

Estes pensamentos, podem ser descritos como uma seqüência de instruções, que devem ser seguidas para se cumprir uma determinada tarefa.

Seqüência Lógica são passos executados até atingir um objetivo ou solução de um problema.

1.3 *Instruções*

Na linguagem comum, entende-se por instruções "**um conjunto de regras ou normas definidas para a realização ou emprego de algo**".

Em informática, porém, instrução é a informação que indica a um computador uma ação elementar a executar.

Convém ressaltar que uma ordem isolada não permite realizar o processo completo, para isso é necessário um conjunto de instruções colocadas em ordem seqüencial lógica.

Por exemplo, se quisermos fazer uma omelete de batatas, precisaremos colocar em prática uma série de instruções: descascar as batatas, bater os ovos, fritar as batatas, etc...

É evidente que essas instruções tem que ser executadas em uma ordem adequada – não se pode descascar as batatas depois de fritá-las.

Dessa maneira, uma instrução tomada em separado não tem muito sentido; para obtermos o resultado, precisamos colocar em prática o conjunto de todas as instruções, na ordem correta.

Instruções são um conjunto de regras ou normas definidas para a realização ou emprego de algo. Em informática, é o que indica a um computador uma ação elementar a executar.

1.4 Algoritmo

Um algoritmo é formalmente uma seqüência finita de passos que levam a execução de uma tarefa. Podemos pensar em algoritmo como uma receita, uma seqüência de instruções que dão cabo de uma meta específica. Estas tarefas não podem ser redundantes nem subjetivas na sua definição, devem ser claras e precisas.

Como exemplos de algoritmos podemos citar os algoritmos das operações básicas (adição, multiplicação, divisão e subtração) de números reais decimais. Outros exemplos seriam os manuais de aparelhos eletrônicos, como um videocassete, que explicam passo-a-passo como, por exemplo, gravar um evento.

Até mesmo as coisas mais simples, podem ser descritas por seqüências lógicas. Por exemplo:

"Chupar uma bala".

- Pegar a bala
- Retirar o papel
- Chupar a bala
- Jogar o papel no lixo

"Somar dois números quaisquer".

- Escreva o primeiro número no retângulo A
- Escreva o segundo número no retângulo B
- Some o número do retângulo A com número do retângulo B e coloque o resultado no retângulo C

1.5 Programas

Os programas de computadores nada mais são do que algoritmos escritos numa linguagem de computador (Pascal, C, Cobol, Fortran, Visual Basic entre outras) e que são interpretados e executados por uma máquina, no caso um computador. Notem que dada esta interpretação rigorosa, um programa é por natureza muito específico e rígido em relação aos algoritmos da vida real.

2.1 Pseudocódigo

Os algoritmos são descritos em uma linguagem chamada **pseudocódigo**. Este nome é uma alusão à posterior implementação em uma linguagem de programação, ou seja, quando formos programar em uma linguagem, por exemplo Visual Basic, estaremos gerando código em Visual Basic. Por isso os algoritmos são independentes das linguagens de programação. Ao contrário de uma linguagem de programação não existe um formalismo rígido de como deve ser escrito o algoritmo.

O algoritmo deve ser fácil de se interpretar e fácil de codificar. Ou seja, ele deve ser o intermediário entre a linguagem falada e a linguagem de programação.

2.2 Regras para construção do Algoritmo

Para escrever um algoritmo precisamos descrever a seqüência de instruções, de maneira simples e objetiva. Para isso utilizaremos algumas técnicas:

- Usar somente um verbo por frase
- Imaginar que você está desenvolvendo um algoritmo para pessoas que não trabalham com informática
- Usar frases curtas e simples
- Ser objetivo
- Procurar usar palavras que não tenham sentido dúbio

2.3 Fases

No capítulo anterior vimos que ALGORITMO é uma seqüência lógica de instruções que podem ser executadas.

É importante ressaltar que qualquer tarefa que siga determinado padrão pode ser descrita por um algoritmo, como por exemplo:

COMO FAZER ARROZ DOCE

ou então

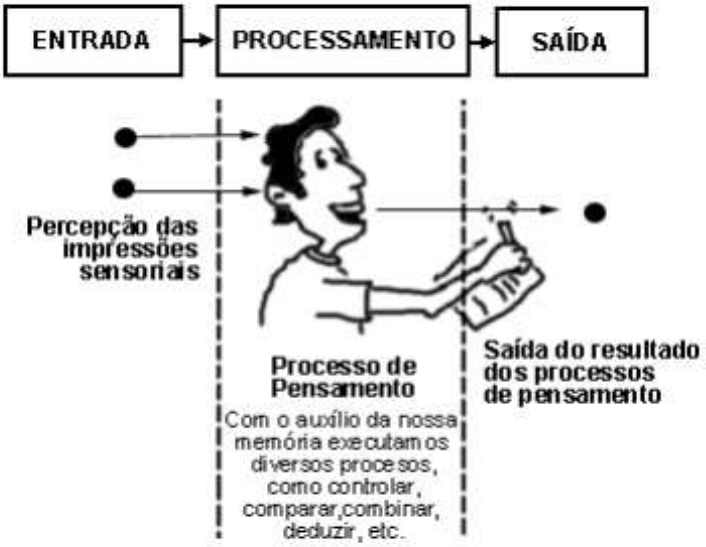
CALCULAR O SALDO FINANCEIRO DE UM ESTOQUE

Entretanto ao montar um algoritmo, precisamos primeiro dividir o problema apresentado em três fases fundamentais.

Onde temos:

ENTRADA: São os dados de entrada do algoritmo
PROCESSAMENTO: São os procedimentos utilizados para chegar ao resultado final
SAÍDA: São os dados já processados

Analogia com o homem



2.4 *Exemplo de Algoritmo*

Imagine o seguinte problema: Calcular a média final dos alunos da 3ª Série. Os alunos realizarão quatro provas: P1, P2, P3 e P4.

Onde:

Média Final = $\frac{P1 + P2 + P3 + P4}{4}$

Para montar o algoritmo proposto, faremos três perguntas:

a) Quais são os dados de entrada?

R: Os dados de entrada são P1, P2, P3 e P4

b) Qual será o processamento a ser utilizado?

R: O procedimento será somar todos os dados de entrada e dividi-los por 4 (quatro)

$$\frac{P1 + P2 + P3 + P4}{4}$$

c) Quais serão os dados de saída?

R: O dado de saída será a média final

Algoritmo

Receba a nota da prova1
Receba a nota de prova2
Receba a nota de prova3
Receba a nota da prova4
Some todas as notas e divida o resultado por 4
Mostre o resultado da divisão

2.5 *Teste de Mesa*

Após desenvolver um algoritmo ele deverá sempre ser testado. Este teste é chamado de ***TESTE DE MESA***, que significa, seguir as instruções do algoritmo de maneira precisa para verificar se o procedimento utilizado está correto ou não.

Veja o exemplo:

Nota da Prova 1
Nota da Prova 2
Nota da Prova 3
Nota da Prova 4

Utilize a tabela abaixo:

P1	P2	P3	P4	Média

3 Diagrama de Bloco

3.1 *O que é um diagrama de bloco?*

O diagrama de blocos é uma forma padronizada e eficaz para representar os passos lógicos de um determinado processamento.

Com o diagrama podemos definir uma seqüência de símbolos, com significado bem definido, portanto, sua principal função é a de facilitar a visualização dos passos de um processamento.

3.2 *Simbologia*

Existem diversos símbolos em um diagrama de bloco. No decorrer do curso apresentaremos os mais utilizados.

Veja no quadro abaixo alguns dos símbolos que iremos utilizar:

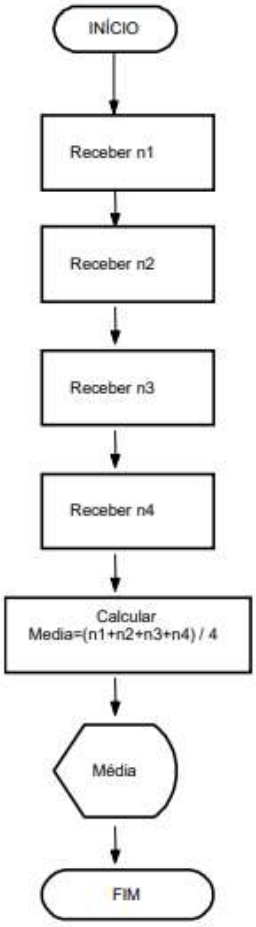
Símbolo	**Função**
TERMINAL	Indica o INÍCIO ou FIM de um processamento Exemplo: Início do algoritmo
PROCESSAMENTO	Processamento em geral Exemplo: Calculo de dois números
ENTRADA DE DADO MANUAL	Indica entrada de dados através do Teclado Exemplo: Digite a nota da prova 1
EXIBIR	Mostra informações ou resultados Exemplo: Mostre o resultado do calculo

Dentro do símbolo sempre terá algo escrito, pois somente os símbolos não nos dizem nada. Veja no exemplo a seguir:

Exemplos de Diagrama de Bloco

"CHUPAR UMA BALA"

"CALCULAR A MÉDIA DE 4 NOTAS



Veja que no exemplo da bala seguimos uma seqüência lógica somente com informações diretas, já no segundo exemplo da média utilizamos cálculo e exibimos o resultado do mesmo.

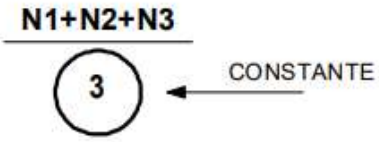
4 Constantes, Variáveis e Tipos de Dados

Variáveis e constantes são os elementos básicos que um programa manipula. Uma variável é um espaço reservado na memória do computador para armazenar um tipo de dado determinado. Variáveis devem receber nomes para poderem ser referenciadas e modificadas quando necessário. Um programa deve conter declarações que especificam de que tipo são as variáveis que ele utilizará e as vezes um valor inicial. Tipos podem ser por exemplo: inteiros, reais, caracteres, etc. As expressões combinam variáveis e constantes para calcular novos valores.

4.1 Constantes

Constante é um determinado valor fixo que não se modifica ao longo do tempo, durante a execução de um programa. Conforme o seu tipo, a constante é classificada como sendo numérica, lógica e literal.

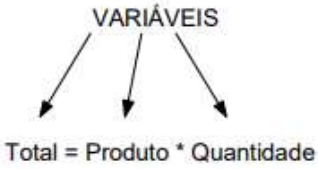
Exemplo de constantes:

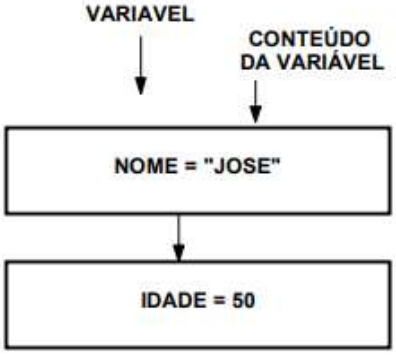


4.2 Variáveis

Variável é a representação simbólica dos elementos de um certo conjunto. Cada variável corresponde a uma posição de memória, cujo conteúdo pode se alterado ao longo do tempo durante a execução de um programa. Embora uma variável possa assumir diferentes valores, ela só pode armazenar um valor a cada instante

Exemplos de variáveis





4.3 Tipos de Variáveis

As variáveis e as constantes podem ser basicamente de quatro tipos: Numéricas, caracteres, Alfanuméricas ou lógicas.

Numéricas Específicas para armazenamento de números, que posteriormente poderão ser utilizados para cálculos. Podem ser ainda classificadas como Inteiras ou Reais. As variáveis do tipo inteiro são para armazenamento de números inteiros e as Reais são para o armazenamento de números que possuam casas decimais.

Caracteres Específicas para armazenamento de conjunto de caracteres que não contenham números (literais). Ex: nomes.

Alfanuméricas Específicas para dados que contenham letras e/ou números. Pode em determinados momentos conter somente dados numéricos ou somente literais. Se usado somente para armazenamento de números, não poderá ser utilizada para operações matemáticas.

Lógicas Armazenam somente dados lógicos que podem ser Verdadeiro ou Falso.

4.4 Declaração de Variáveis

As variáveis só podem armazenar valores de um mesmo tipo, de maneira que também são classificadas como sendo numéricas, lógicas e literais.

5 Operadores

Os operadores são meios pelo qual incrementamos, decrementamos, comparamos e avaliamos dados dentro do computador. Temos três tipos de operadores:

- Operadores Aritméticos
- Operadores Relacionais
- Operadores Lógicos

5.1 Operadores Aritméticos

Os operadores aritméticos são os utilizados para obter resultados numéricos. Além da adição, subtração, multiplicação e divisão, podem utilizar também o operador para exponenciação. Os símbolos para os operadores aritméticos são:

OPERAÇÃO	SÍMBOLO
Adição	+
Subtração	-
Multiplicação	*
Divisão	/
Exponenciação	**

5.1 *Operadores Aritméticos*

Os operadores aritméticos são os utilizados para obter resultados numéricos. Além da adição, subtração, multiplicação e divisão, podem utilizar também o operador para exponenciação. Os símbolos para os operadores aritméticos são:

OPERAÇÃO	SÍMBOLO
Adição	+
Subtração	-
Multiplicação	*
Divisão	/
Exponenciação	**

Hierarquia das Operações Aritméticas

1 º ( ) Parênteses
2 º Exponenciação
3 º Multiplicação, divisão (o que aparecer primeiro)
4 º + ou – (o que aparecer primeiro)

Exemplo

TOTAL = PRECO * QUANTIDADE

1 + 7 * 2 ** 2 –1 = 28

3 * (1 – 2) + 4 * 2 = 5

5.2 *Operadores Relacionais*

Os operadores relacionais são utilizados para comparar ***String*** de caracteres e números. Os valores a serem comparados podem ser caracteres ou variáveis.

Estes operadores sempre retornam valores lógicos (verdadeiro ou falso/ True ou False)

Para estabelecer prioridades no que diz respeito a qual operação executar primeiro, utilize os parênteses.

Os operadores relacionais são:

Descrição	Símbolo
Igual a	=
Diferente de	<> ou #
Maior que	>
Menor que	<
Maior ou igual a	>=
Menor ou igual a	<=

Exemplo:

Tendo duas variáveis A = 5 e B = 3

Os resultados das expressões seriam:

Expressão	Resultado
A = B	Falso
A <> B	Verdadeiro
A > B	Verdadeiro
A < B	Falso
A >= B	Verdadeiro
A <= B	Falso

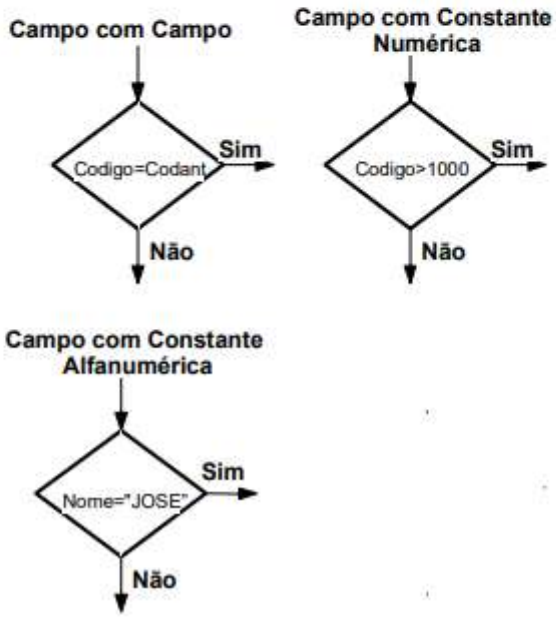
Exemplo:

Tendo duas variáveis A = 5 e B = 3

Os resultados das expressões seriam:

Expressão	Resultado
A = B	Falso
A <> B	Verdadeiro
A > B	Verdadeiro
A < B	Falso
A >= B	Verdadeiro
A <= B	Falso

Símbolo Utilizado para comparação entre expressões



5.3 Operadores Lógicos

Os operadores lógicos servem para combinar resultados de expressões, retornando se o resultado final é verdadeiro ou falso.

Os operadores lógicos são:

E	AND
OU	OR
NÃO	NOT

E / AND — Uma expressão AND (E) é verdadeira se todas as condições forem verdadeiras

OR/OU — Uma expressão OR (OU) é verdadeira se pelo menos uma condição for verdadeira

NOT — Um expressão NOT (NÃO) inverte o valor da expressão ou condição, se verdadeira inverte para falsa e vice-versa.

A tabela abaixo mostra todos os valores possíveis criados pelos três operadores lógicos (AND, OR e NOT)

1º Valor	Operador	2º Valor	Resultado
T	AND	T	T
T	AND	F	F
F	AND	T	F
F	AND	F	F
T	OR	T	T
T	OR	F	T
F	OR	T	T
F	OR	F	F
T	NOT		F
F	NOT		T

Exemplos:

Suponha que temos três variáveis A = 5, B = 8 e C =1

Os resultados das expressões seriam:

Expressões			Resultado
A = B	AND	B > C	Falso
A <> B	OR	B < C	Verdadeiro
A > B	NOT		Verdadeiro
A < B	AND	B > C	Verdadeiro
A >= B	OR	B = C	Falso
A <= B	NOT		Falso

6 Operações Lógicas

Operações Lógicas são utilizadas quando se torna necessário tomar decisões em um diagrama de bloco.

Num diagrama de bloco, toda decisão terá sempre como resposta o resultado VERDADEIRO ou FALSO.

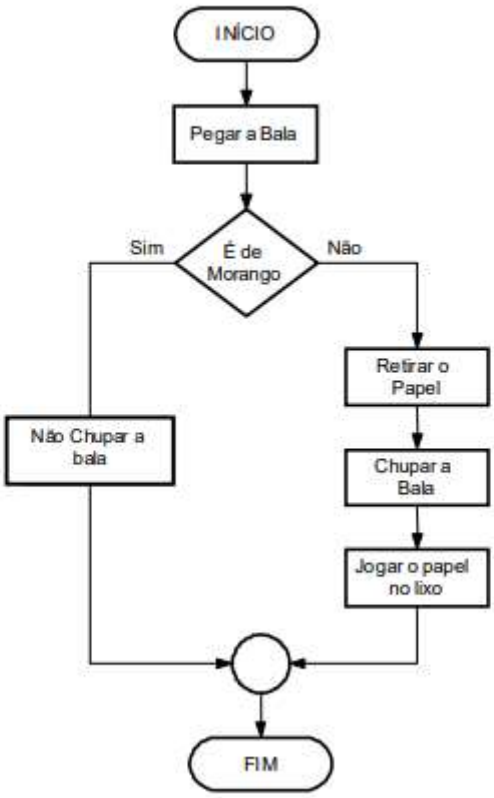
Como no exemplo do algoritmo "CHUPAR UMA BALA". Imaginemos que algumas pessoas não gostem de chupar bala de Morango, neste caso teremos que modificar o algoritmo para:

"Chupar uma bala".

- Pegar a bala
- A bala é de morango?
    - Se **sim**, não chupe a bala
    - Se **não**, continue com o algoritmo

- Retirar o papel
- Chupar a bala
- Jogar o papel no lixo

Exemplo: Algoritmo "Chupar Bala" utilizando diagrama de Blocos



7 Estrutura de Decisão e Repetição

Como vimos no capítulo anterior em "Operações Lógicas", verificamos que na maioria das vezes necessitamos tomar decisões no andamento do algoritmo. Essas decisões interferem diretamente no andamento do programa. Trabalharemos com dois tipos de estrutura. A estrutura de Decisão e a estrutura de Repetição

7.1 Comandos de Decisão

Os comandos de decisão ou desvio fazem parte das técnicas de programação que conduzem a estruturas de programas que não são totalmente seqüenciais. Com as instruções de SALTO ou DESVIO pode-se fazer com que o programa proceda de uma ou outra maneira, de acordo com as decisões lógicas tomadas em função dos dados ou resultados anteriores. As principais estruturas de decisão são: "***Se Então***", "***Se então Senão***" e "***Caso Selecione***"

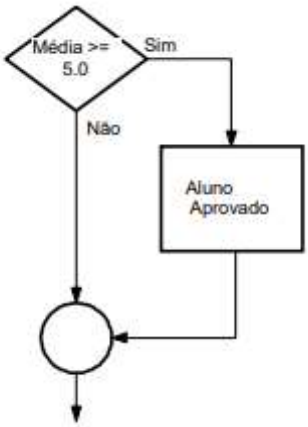
7.1.1 SE ENTÃO / IF ... THEN

A estrutura de decisão "SE/IF" normalmente vem acompanhada de um comando, ou seja, se determinada condição for satisfeita pelo comando SE/IF então execute determinado comando.

Imagine um algoritmo que determinado aluno somente estará aprovado se sua média for maior ou igual a 5.0, veja no exemplo de algoritmo como ficaria.

SE MEDIA >= 5.0 ENTÃO ALUNO APROVADO

Em diagrama de blocos ficaria assim:



Em Visual Basic

```
IF MEDIA >= 5 Then
   Text1 = "APROVADO"
ENDIF
```

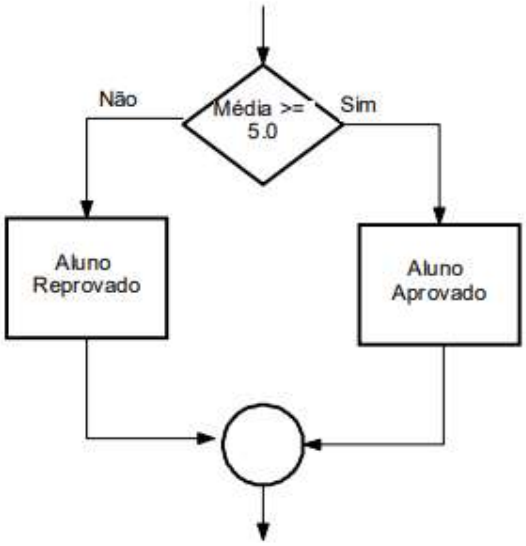
7.1.2 SE ENTÃO SENÃO / IF ... THEN ... ELSE

A estrutura de decisão "SE/ENTÃO/SENÃO", funciona exatamente como a estrutura "SE", com apenas uma diferença, em "SE" somente podemos executar comandos caso a condição seja verdadeira, diferente de "SE/SENÃO" pois sempre um comando será executado independente da condição, ou seja, caso a condição seja "verdadeira" o comando da condição será executado, caso contrário o comando da condição "falsa" será executado

Em algoritmo ficaria assim:

```
SE MÉDIA >= 5.0 ENTÃO
    ALUNO APROVADO
SENÃO
    ALUNO REPROVADO
```

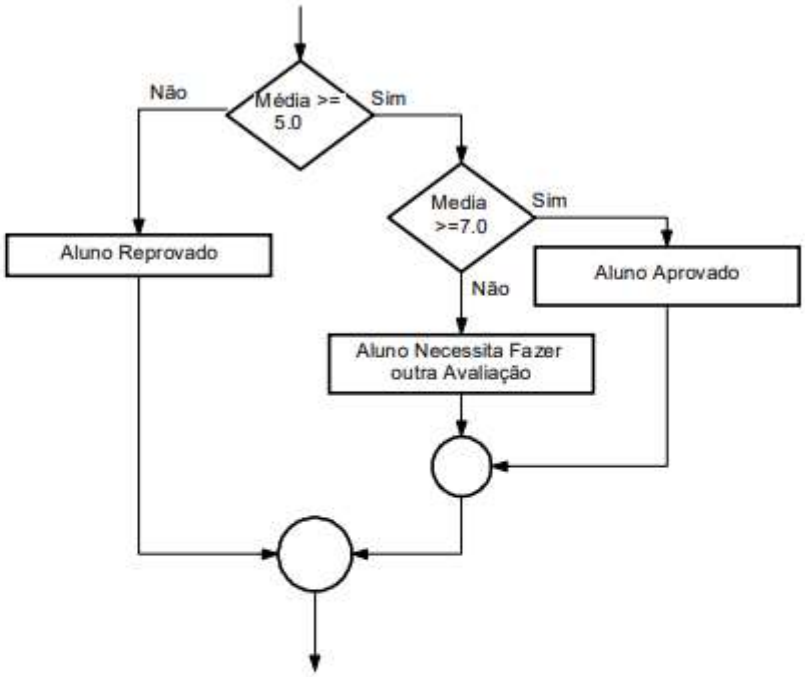
Em diagrama



Em Visual Basic

```
IF MEDIA >= 5 Then
   Text1 = "APROVADO"
ELSE
   Text1 = "REPROVADO"
ENDIF
```

No exemplo acima está sendo executada uma condição que, se for verdadeira, executa o comando "APROVADO", caso contrário executa o segundo comando "REPROVADO". Podemos também dentro de uma mesma condição testar outras condições. Como no exemplo abaixo:



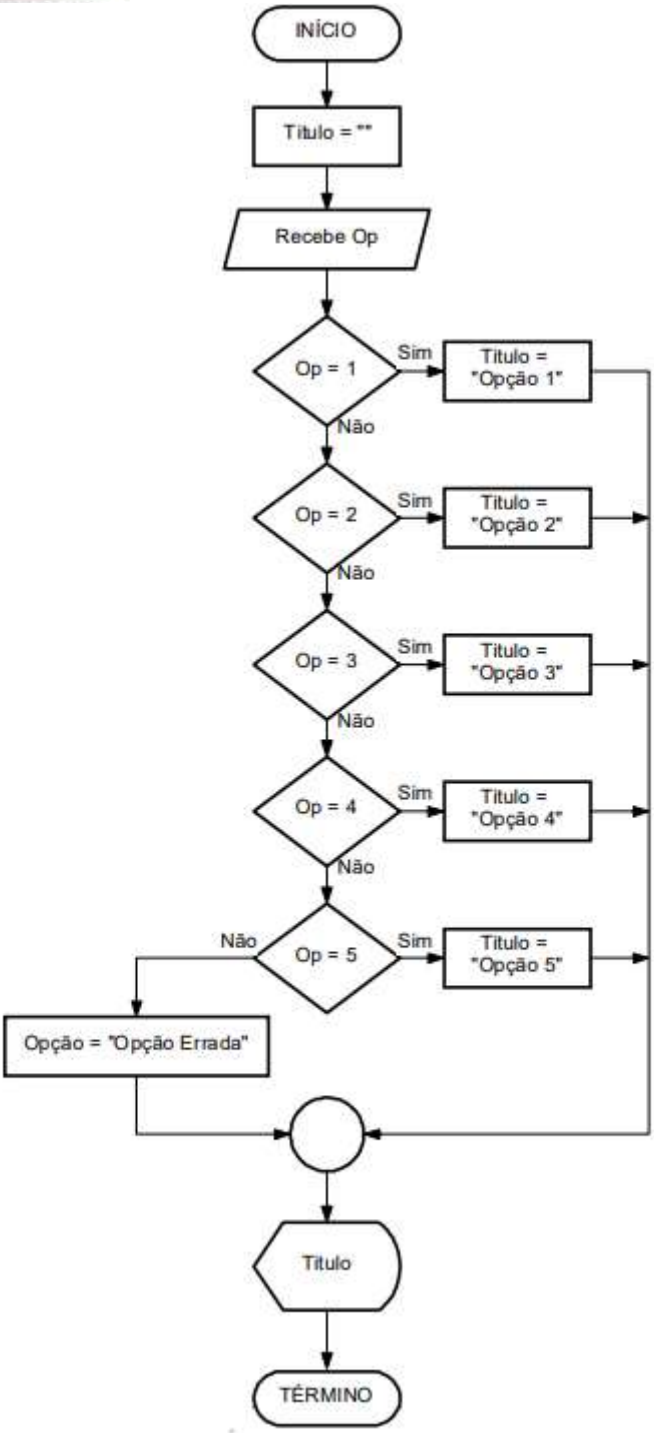
Em Visual Basic

```
IF MEDIA >= 5 Then
   IF MEDIA >= 7.0 then
      Text1 = "Aluno APROVADO"
   ELSE
      Text1 = "Aluno Necessita fazer outra Avaliação"
   ENDIF
ELSE
   Text1 = "Aluno REPROVADO"
ENDIF
```

7.1.3 CASO SELECIONE / SELECT ... CASE

A estrutura de decisão CASO/SELECIONE é utilizada para testar, na condição, uma única expressão, que produz um resultado, ou, então, o valor de uma variável, em que está armazenado um determinado conteúdo. Compara-se, então, o resultado obtido no teste com os valores fornecidos em cada cláusula "Caso".

No exemplo do diagrama de blocos abaixo, é recebido uma variável "**Op**" e testado seu conteúdo, caso uma das condições seja satisfeita, é atribuído para a variável Titulo a String "Opção X", caso contrário é atribuído a string "Opção Errada".


INÍCIO
Titulo = ""
Recebe Op
Op = 1
Sim
Titulo = "Opção 1"
Não
Op = 2
Sim
Titulo = "Opção 2"
Não
Op = 3
Sim
Titulo = "Opção 3"
Não
Op = 4
Sim
Titulo = "Opção 4"
Não
Op = 5
Sim
Titulo = "Opção 5"
Não
Opção = "Opção Errada"
Titulo
TÉRMINO

Em Visual Basic utilizamos a seguinte seqüência de comandos para representar o diagrama anterior.

```
TITULO = ""
OP = INPUTBOX("DIGITE A OPÇÃO")
SELECT CASE OP
        CASE 1
            TITULO = "OPÇÃO 1"
        CASE 2
            TITULO = "OPÇÃO 2"
        CASE 3
            TITULO = "OPÇÃO 3"
        CASE 4
            TITULO = "OPÇÃO 4"
        CASE 5
             TITULO = "OPÇÃO 5"
        CASE ELSE
             TITULO = "OPÇÃO ERRADA"
END SELECT

LABEL1.CAPTION = TITULO
```

7.2 *Comandos de Repetição*

Utilizamos os comandos de repetição quando desejamos que um determinado conjunto de instruções ou comandos sejam executados um número definido ou indefinido de vezes, ou enquanto um determinado estado de coisas prevalecer ou até que seja alcançado.

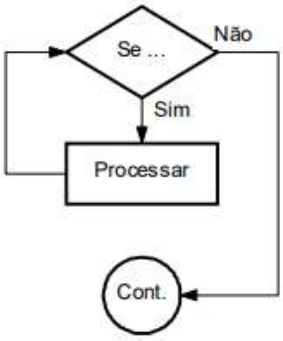
Trabalharemos com modelos de comandos de repetição:

- Enquanto x, processar (**Do While ...Loop**);
- Até que x, processar ... (**Do Until ... Loop**);
- Processar ..., Enquanto x (**Do ... Loop While**);
- Processar ..., Até que x (**Do ... Loop Until**)
- Para ... Até ... Seguinte (**For ... To ... Next**)

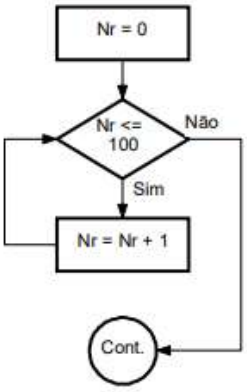
7.2.1 Enquanto x, Processar (Do While ... Loop)

Neste caso, o bloco de operações será executado enquanto a condição x for verdadeira. O teste da condição será sempre realizado antes de qualquer operação. Enquanto a condição for verdadeira o processo se repete. Podemos utilizar essa estrutura para trabalharmos com contadores.

Em diagrama de bloco a estrutura é a seguinte:



Exemplo de Contador



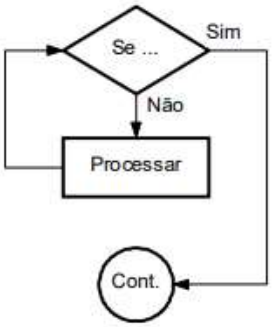
Em Visual Basic:

```
Nr = 0
Do While Nr <= 100
   Nr = Nr + 1
Loop
```

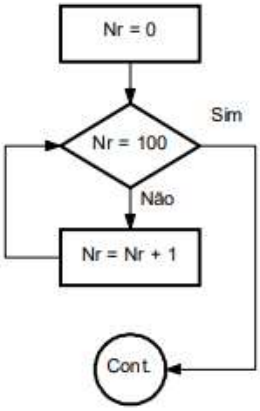
7.2.2 Até que x, processar ... (Do Until ... Loop)

Neste caso, o bloco de operações será executado até que a condição seja satisfeita, ou seja, somente executará os comandos enquanto a condição for falsa.

Em diagrama de bloco



Exemplo de Até Diagrama



Em Visual Basic

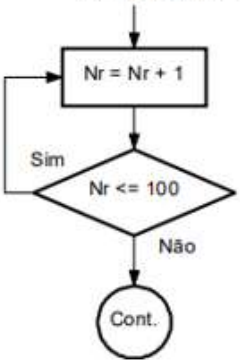
```
Nr = 0
Do Until Nr = 100
    Nr = Nr + 1
Loop

Label1.Caption = Nr
```

7.2.3 Processar ..., Enquanto x (Do ... Loop While)

Neste caso primeiro são executados os comandos, e somente depois realizado o teste da condição. Se a condição for verdadeira, os comandos si executados novamente, caso seja falso é encerrado o comando DO.

Em diagrama de bloco

Exemplo de Até Diagrama



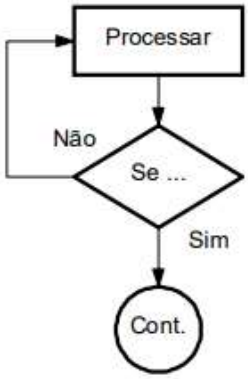
Em Visual Basic

```
Nr = 0
Do
    Nr = Nr + 1
Loop While Nr <= 100

Label1.Caption = Nr
```

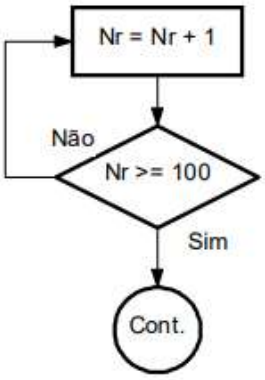
7.2.4 Processar ..., Até que x (Do ... Loop Until)

Neste caso, executa-se primeiro o bloco de operações e somente depois é realizado o teste de condição. Se a condição for verdadeira, o fluxo do programa continua normalmente. Caso contrário é processado novamente os comandos antes do teste da condição.

Em diagrama de Bloco



Exemplo de Do .... Loop - Until



Em Visual Basic

```
nr = 0
Do
   nr = nr + 1
Loop Until nr >= 100
Label1.Caption = nr
```

8 Arquivos de Dados

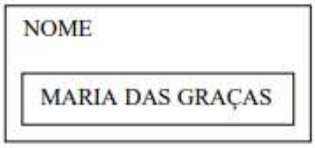
Os dados manipulados até o momento, estavam em memória, ou seja, após a execução do diagrama os dados se perdiam. Para resolver esse problema começaremos a trabalhar com arquivos, onde poderemos guardar os dados e também manipula-los. Para isso necessitamos rever alguns conceitos como: campos, registros e arquivos.

8.1 Conceitos Básicos

CAMPO é um espaço reservado em memória para receber informações (dados).

Exemplo: Campo Nome, Campo Endereço

Campo na memória



REGISTRO é um conjunto de campos

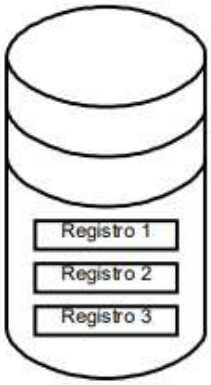
Exemplo: Registro de Clientes

COD-CLI	NOME	ENDEREÇO	FONE
00001	MARIA DAS GRAÇAS	RUA DAS DORES,1400	888-9876

ARQUIVO é um conjunto de registros

Exemplo: O arquivo de Clientes da Empresa, onde estão armazenados os dados de todos os clientes da empresa.

ARQ-CLI



8.2 Abertura de Arquivos

Toda vez que for necessário trabalhar com arquivo, primeiramente precisamos **ABRIR** o arquivo. Abrir o arquivo significa alocar o periférico (disco, disquete) em que o arquivo se encontra, e deixá-lo disponível para leitura/gravação.

O símbolo para abertura de arquivo

8.3 Fechamento de Arquivos

Da mesma maneira que precisamos abrir um arquivo antes do processamento, também se faz necessário o fechamento do mesmo, para que suas informações não possam ser violadas ou danificadas.

Fechar um arquivo significa liberar o periférico que estava sendo utilizado.

O símbolo para fechamento de arquivo

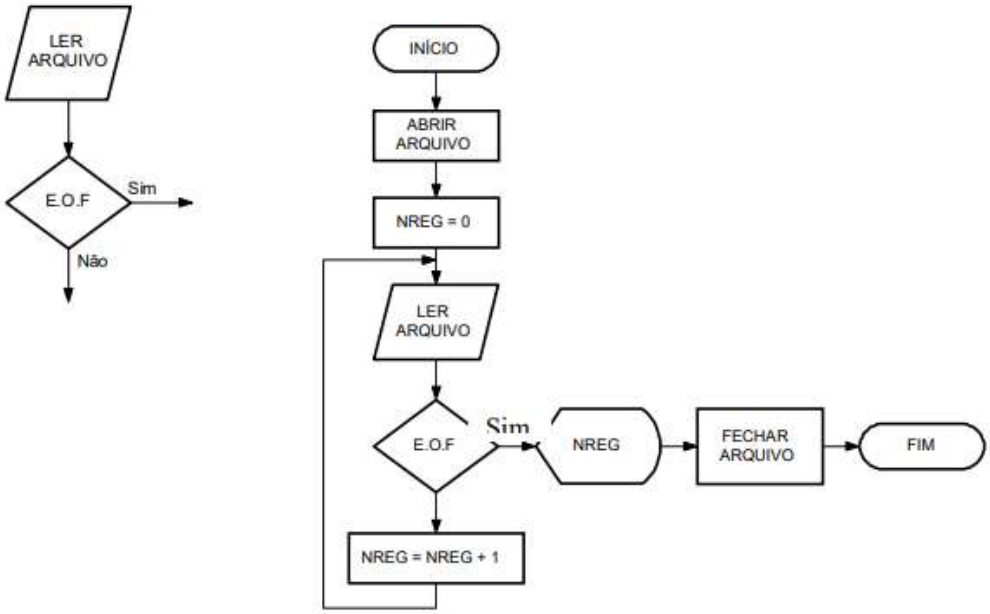
8.4 Leitura de Arquivos

Após abrir um arquivo é necessário **LER** os dados que estão em disco e transferi-los para memória. Essa transferência é feita por registro. Esse procedimento é gerenciado pelo próprio sistema operacional.

O símbolo para leitura de arquivo

Toda vez que abrimos um arquivo ele posiciona o "**ponteiro**" no primeiro registro, ou seja, no início do arquivo. Para que possamos trabalhar com os dados se torna necessário sabermos onde está o ponteiro do registro. Isso poderemos fazer testando se o ponteiro está no início (**BOF – Bottom Of File**) ou no final do arquivo (**EOF – End Of File**). Esse é sempre executado após a leitura do registro (mudança da posição do ponteiro). Simbolicamente podemos representar esse passo da seguinte maneira.

Exemplo de diagrama de bloco



8.5 Movimentação de registros

Como dito no item anterior, quando um arquivo é aberto o ponteiro está no primeiro registro. A cada leitura do Arquivo o ponteiro se movimenta para o próximo registro e assim por diante. Como mostra a figura abaixo:

COD-CLI	NOME	ENDERECO	TELEFONE
00001	JOSE DA SILVA	RUA DAS CAMELIAS	788-2000
00002	MARIA	RUA DAS DORES	788-2001
00003	ANTONIO	RUA HORTENCIAS	788-2002

8.6 Gravação de Arquivos

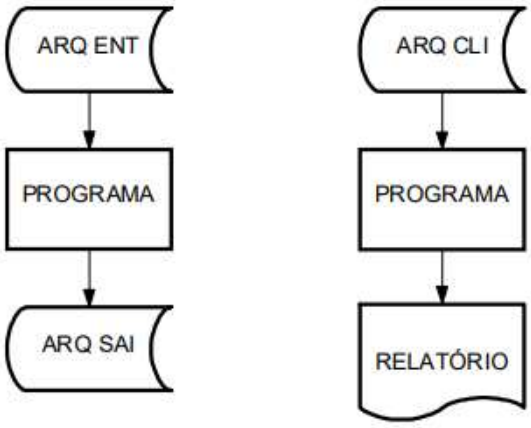
Da mesma maneira que os registros são lidos de um arquivo, também devemos gravar registros em um arquivo.

A gravação consiste na transferência de um registro da memória, para um periférico (disco, disquete).

O símbolo para gravação de arquivos

8.7 Macro Fluxo

O macro fluxo é a representação gráfica dos arquivos que serão processados em um programa.



Estes dois exemplos de Macro-fluxo dão uma visão geral de como devemos proceder com cada um dos programas. O primeiro diz que haverá um arquivo de entrada, um processamento e um arquivo de saída. Já o segundo exemplo diz que haverá um arquivo de entrada, um processamento, e a saída serão um relatório.

9 Relatórios

A impressão de relatórios é o registro de informações processadas pelo computador em um meio de armazenamento de dados chamado de formulário. Para efetuarmos a impressão de relatórios devemos nos preocupar com os seguintes aspectos:

- Características do formulário
- Controle de linhas e salto de página
- Impressão de cabeçalho e estética da página
- Impressão de rodapé
- Numeração de páginas

9.1 Características do Formulário

A maioria dos formulários possui um formato padrão, isto é, a quantidade de linhas por página e de caracteres por linha são constantes.

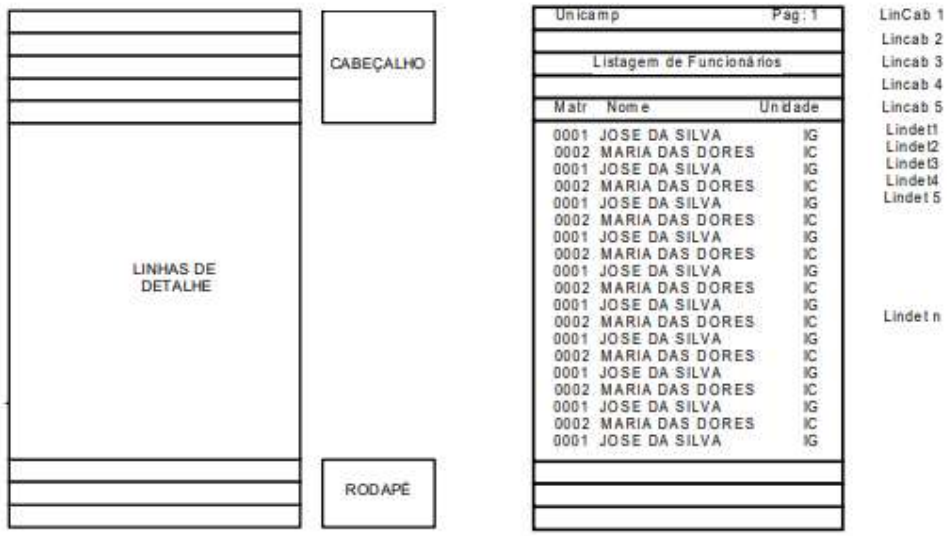
9.2 Controle de linhas e salto de páginas

Uma preocupação com impressão de relatórios é não permitir que a impressora imprima fora do papel, pois além de esteticamente não ficar bom, haveria perda de informações.

Para controlarmos o número de linhas impressas, devemos criar um contador de linha e não deixar o valor desses contadores ultrapassarem o número desejado de linhas por páginas.

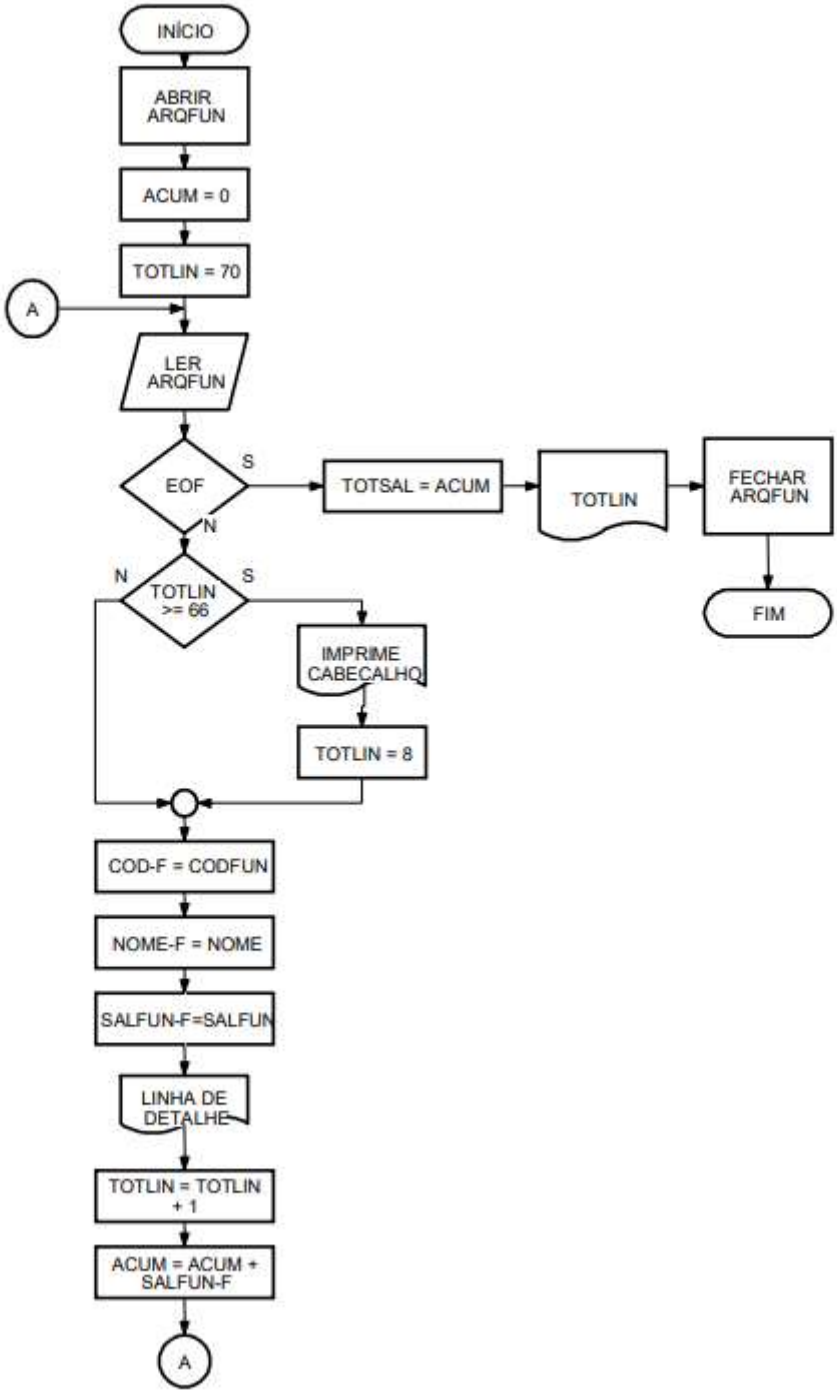
9.3 Impressão de Cabeçalho e Estética de Página

Para termos uma idéia melhor da estética do formulário, veja o exemplo abaixo.



Área de Cabeçalho	Local onde devemos colocar um cabeçalho para identificarmos o assunto a que se refere o conteúdo da página como um todo, e um cabeçalho indicando o significado do conteúdo de cada coluna de informações. Pode haver outras linhas de cabeçalho de acordo com a necessidade.
Linha de Detalhe	São as linhas geradas a partir de dados lidos de um arquivo.
Área de Rodapé	Pode haver linhas contendo valores de totalizações de determinadas colunas e/ou linhas de identificação da empresa, ou outras informações qualquer.

Veja abaixo um exemplo de diagrama de bloco para impressão de relatório



10 Simbologia

Símbolo	Função
TERMINAL	Indica o INÍCIO ou FIM de um processamento Exemplo: Início do algoritmo
PROCESSAMENTO	Processamento em geral Exemplo: Calculo de dois números
ENTRA/SAÍDA	Operação de entrada e saída de dados Exemplo: Leitura e Gravação de Arquivos
DECISÃO	Indica uma decisão a ser tomada Exemplo: Verificação de Sexo
DESVIO	Permite o desvio para um ponto qualquer do programa
ENTRADA MANUAL	Indica entrada de dados através do Teclado Exemplo: Digite a nota da prova 1
EXIBIR	Mostra informações ou resultados Exemplo: Mostre o resultado do calculo
RELATÓRIO	Relatórios

REFERÊNCIAS

CENTRO PAULA SOUZA. **Informática, programação de computadores** (Vol. 4). São Paulo: Fundação Padre Anchieta, 2010.

DEITEL, P., & DEITEL, H. **Java - Como Programar**. São Paulo, SP: Pearson Prentice Hall, 2010.

PUGA, S., & RISSETTI, G. **Lógica de programação e estrutura de dados com aplicações em Java.** São Paulo: Pearson Prentice Hall, 2009.

SCHILDT, H. **Java para iniciantes.** Porto Alegre: Bookman, 2015.

FORBELLONE, André L. V.; ELBERSPACHER, Henri Frederico. **Lógica de Programação:** a Construção de Algoritmos e Estrutura de Dados. Editora Pearson, 2000.

MANZANO, José Augusto N. G; OLIVEIRA, Jayr Figueiredo. **Algoritmos:** Lógica para Desenvolvimento de Programação. Editora Érica, 2007.

MORAES, Paulo Sérgio de. **"Lógica de Programação",** 2000, disponível em:< http://www.siban.com.br/destaque/21_carta.pdf>. Acessado em 29 de março de 2022.

PUGA, SANDRA; RISSETTI, GERSON. **Lógica de Programação e Estruturas de Dados com Aplicações em Java**. Editora Pearson. 2009

SCHILDT, HEBERT. **Java para Iniciantes.** Editora Bookman.2015.